

Mohammad Ali Jinnah University

Department of Electrical & Computer Engineering

Computer Vision Based Parking Authorization & Authentication System

Team Members:

Hamza Aijaz (FA22-BECE-0023)

Mahmood Badar (FA22-BECE-0027)

Supervised By:

Engr. Muhammad Sufyan

A project report submitted to the Department of Electrical & Computer Engineering, Faculty of Engineering, Mohammad Ali Jinnah University in partial fulfillment of the degree of Bachelor of Engineering in Computer System Engineering

SPRING 2026

Acknowledgment

All praises are due to **Allah Almighty**, the Most Gracious and the Most Merciful, who granted us the strength, knowledge, and patience to complete this Final Year Project. Without His countless blessings and guidance, this milestone would not have been possible.

We want to thank our respected supervisor, Engr. Muhammad Sufyan, for his unwavering support, insightful suggestions, and constructive feedback throughout the project. His expert supervision kept us focused and motivated at every stage, from ideation to execution.

Our sincere thanks to all the respected faculty members of the Department of Computer System Engineering, Mohammad Ali Jinnah University, who equipped us with the knowledge and skills that laid the foundation for this work. The academic environment and learning resources provided by the university were instrumental in developing our technical and research capabilities.

We are thankful to our classmates, friends, and teammates for constant support, collaboration, and idea sharing, which enriched our project experience. Special thanks to our group members for their dedication, coordination, and mutual respect throughout this journey.

We cannot thank our families enough for their unconditional love, emotional support, and patience. Their belief in us, even during times of stress and pressure, kept us going.

Lastly, we would like to acknowledge all online communities, technical forums, and researchers whose work inspired and guided our efforts, and everyone else who contributed success of this project.

Certificate of Approval



It is certified that the Project titled **"Computer Vision Based Parking Authorization & Authentication System"** has been completed by **Hamza Aijaz & Mahmood Badar**, under the supervision of Engr. Muhammad Sufyan.

No part of this project has been submitted previously for the award of any degree at any other institution.

This project is submitted to the Faculty of Engineering, Mohammad Ali Jinnah University, Karachi, Pakistan, in partial fulfillment of the requirements for the degree of Bachelor of Engineering (Spring 2026).

Student Name: Hamza Aijaz Signature: _____

Student Name: Mahmood Badar Signature: _____

_____	_____	_____
Project Coordinator	**HoD (FoE)**	**Program Dean (FoE)**

Certificate of Supervision

This is to certify that the Project titled **"Computer Vision Based Parking Authorization & Authentication System"** has been submitted to the Faculty of Engineering, Semester Spring 2026, by **Hamza Aijaz & Mahmood Badar**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer System Engineering.

The project has been carried out under my supervision. I hereby certify that the work submitted is original and has not been plagiarized from any other source, except where properly cited in the references. Furthermore, this work has not been previously submitted to any other institution for the award of a degree.

Engr. Muhammad Sufyan

Supervisor's Name

Supervisor's Signature

Statement of Originality

Program	BE – CSE	
Student Name	Hamza Aijaz	Mahmood Badar
Student ID	FA22-BECE-0023	FA22-BECE-0027
Email	hamza.aijaz6143@gmail.com	mahmoodbadariqbal@gmail.com
Mobile No.	0312-8345456	0334-3313832

Project Title:

Computer Vision Based Parking Authorization & Authentication System

I/ We do solemnly and sincerely declare that:

I. We are the authors of this work.

II. This work is original.

III. We have sought all necessary departmental permissions governing data collection from the organization concerned.

IV. I have acquired official documents that demonstrate my participation/association in the project that I undertook as part of my Final Year Project.

V. Use of any work in which copyright exists has been made by way of fair dealing and for permitted purposes and any excerpt or extract from, or reference to or reproduction of any copyright work has been explicitly and sufficiently acknowledged. The title of the work and its authorship have been properly cited in this report. I do not have any actual knowledge nor do I ought reasonably to know that the making of this work constitutes an infringement of any copyright work.

VI. I hereby assign all and every right in copyright of this work to Mohammad Ali Jinnah University (MAJU), who henceforth shall be owner of the copyright in this Work and that any reproduction or use in any form or by any means whatsoever is prohibited without the written consent of MAJU having been first had and obtained.

VII. I am fully aware that if while making this work, I have infringed any copyright whether intentionally or otherwise. I may be subject to legal action, or any other action as may be determined by MAJU.

_____ _____

Hamza Aijaz Mahmood Badar

Contents

Acknowledgment ..... **ii**

Certificate of Approval ..... **iii**

Certificate of Supervision ..... **v**

Statement of Originality ..... **vi**

Plagiarism Verification **63**

Contents ..... **vii**

List of Tables ..... **x**

List of Figures ..... **xi**

Abstract ..... **xii**

1 1

Introduction 1

1.1 Background **3**

1.2 Motivation **4**

1.3 Problem Statement **5**

1.3.1 Complex Engineering Problem 7

1.4 Contributions **9**

1.4.1 Relation with the Sustainable Development Goals 10

2 13

Literature Review 13

2.1 Traditional Access Control & RFID Implementation **13**

2.2 Advancement in Automated License Plate Recognition (ALPR) & Computer Vision 16

2.3 Addressing the Passback Security Vulnerability & Spatial Tracking **18**

2.4 Resource Optimization via Dynamic Sensor Positioning **20**

2.5 Emergency Protocol & Administrative Auditing 22

2.6 Relational Database in Edge Security Architecture 23

2.7 Asynchronous Cyber Physical Communication 24

2.8 Human Machine Interfaces (HMI) in Access Control 25

3 27

System Architecture & Methodology 27

3.1 Embedded Hardware Architecture & Edge Interfacing 28

3.1.1 Dual RFID Subsystem Integration (SPI Protocol) 28

3.1.2 Electromechanical Camera Actuation (PWM) 29

3.1.3 Human Machine Interface via I2C Bus 30

3.1.4 Fail Safe Override Matrix (Digital Multiplexing) 31

3.2 Computational Software Tier & Asynchronous Execution 32

3.2.1 Object Oriented System Encapsulation 32

3.2.2 Asynchronous Event Loop & Non-Blocking Concurrency 33

3.2.3 Strict Execution Ordering & Variable State Management 34

3.3 Relational Database Architecture & Spatial State Management 35

3.3.1 The authorized users Entity & State Persistence 36

3.3.2 The access logs Entity & Administrative Auditing 37

3.3.3 Atomic Transactions & Data Integrity 38

3.4 The Machine Vision Pipeline & Optical Character Recognition 38

3.4.1 Optical Hardware Stabilization & Frame Purging 38

3.4.2 Algorithmic Image Pre-Processing & Noise Reduction 39

3.4.3 Deep Learning Text Extraction via CRNN Architecture 40

3.4.4 Heuristic Post-Processing & String Sanitization 41

3.5 The Asynchronous Serial Communication Protocol 42

3.5.1 Baud Rate Optimization & Serial Instantiation 42

3.5.2 Deterministic Payload Structuring from the Edge 43

3.5.3 Micro Byte Command Execution from the Host 44

3.5.4 Buffer Saturation Management & Pipeline Flushing 45

3.6 System Integration & Operational Workflow 45

3.6.1 System Initialization & State Handshake 46

3.6.2 The Master Asynchronous Polling Loop 46
3.6.3 The Dual Factor Verification Sequence 46
3.6.4 Actuation, Feedback, & Audit Logging 48
4 49
Results, Testing, & Evaluation 49
4.1 System Testing Methodology & Environmental Setup 49
4.2 Hardware Integration & Asynchronous Latency Evaluation 50
4.2.1 Serial Communication Throughput 50
4.2.2 Asynchronous Settling Delays & Actuation 51
4.3 Optical Character Recognition (OCR) Efficacy & Algorithmic Resilience 52
4.3.1 Validation of the OpenCV Pre-Processing Pipeline 52
4.3.2 Deep Learning Extraction & CRNN Performance 53
4.3.3 Heuristic Sanitization & Regional Edge Case Mitigation 53
4.4 Spatial Database Integrity & Anti-Passback Validation 54
4.4.1 Rapid Consecutive Authentication Under Load 55
4.4.2 Enforcing Spatial Logic & Exploitation Mitigation 55
5 57
Conclusion & Future Work 57
5.1 Conclusion 57
5.2 Future Recommendations 59
5.2.1 Distributed Database Architecture and Relational Scaling 59
5.2.2 Predictive Analytics and Supervised Machine Learning 59
5.2.3 Hardware Migration to a Unified Edge Appliance 60
5.2.4 Upgrading the Deep Learning Vision Architecture 60
6 61
References 61

List of Tables

Table 1: Complex Engineering Problem 8
Table 2: SDGs table 11
Table 3: End-to-end system latency breakdown, detailing the microsecond hardware communication speeds versus the mechanical and computational processing delays. 51
Table 4: Comparative analysis of EasyOCR extraction accuracy between raw optical matrices and mathematically optimized frames using the OpenCV pre-processing pipeline. 53
Table 5: Performance metrics of the localized deep learning extraction subsystems and post-processing algorithmic sanitization filters. 54
Table 6: Empirical results of the localized SQLite database stress test, detailing transactional integrity and spatial logic performance during rapid consecutive multi-factor authentications. 55

List of Figures

Figure 1-1 2
Figure 1-2 7
Figure 2-1 15
Figure 2-2 17
Figure 2-3 19
Figure 2-4 21
Figure 2-5 23
Figure 3-1 27
Figure 3-2 29
Figure 3-3 29
Figure 3-4 30
Figure 3-5 31
Figure 3-6 33
Figure 3-7 34
Figure 3-8 35
Figure 3-9 36
Figure 3-10 37
Figure 3-11 39
Figure 3-12 40
Figure 3-13 42
Figure 3-14 44
Figure 3-15 44
Figure 3-16 45
Figure 3-17 48

Abstract

This thesis presents the design, implementation, and empirical evaluation of an automated, dual-mode smart parking access control system integrating Radio Frequency Identification (RFID), Optical Character Recognition (OCR), and Anti-Passback logic. To address the security blind spots and unauthorized credential sharing inherent in traditional single-factor authentication models, the proposed cyber-physical architecture combines a centralized edge microcontroller, twin RFID transceivers for bi-directional tracking, a dynamic servo-actuated camera mount, and a dedicated processing core utilizing advanced machine vision frameworks.

Upon an RFID scan at either an entry or exit point, the system executes an automated spatial alignment sequence to physically target the optical sensor toward the vehicle's license plate. To mitigate mechanical vibration and frame lag, the software applies a multi-stage frame-flushing algorithm and a localized digital zoom before submitting the stabilized images to a noise-filtered, adaptive thresholding pipeline. The extracted alphanumeric text is strictly sanitized to filter out extraneous regional identifiers and is subsequently cross-referenced against a localized relational database. This database links user credentials with real-time spatial tracking markers, powering a deterministic Anti-Passback engine that enforces physical access constraints by denying entry or egress attempts that violate structural traffic flow. Furthermore, the architecture features a fully auditable manual override module utilizing a hardware matrix keypad and digital display for emergency access. System evaluation demonstrates high verification accuracy, rapid edge processing latency, robust dynamic camera orientation, and the effective elimination of passback violations under variable environmental conditions.

1

Introduction

The rapid urbanization and increasing density of vehicular traffic in modern educational and corporate environments necessitate highly efficient and secure access control mechanisms. Traditional parking management systems typically rely on isolated authentication methods, such as manual logging by security personnel or standalone Radio Frequency Identification (RFID) tokens. While RFID provides a rapid means of access, it introduces significant security vulnerabilities, most notably "passback" violations where a single authorized credential is inappropriately passed to a second driver to gain illicit entry. To mitigate these vulnerabilities, this project introduces a robust, dual factor Automated Parking Access Control System (PACS) that synchronizes embedded hardware with advanced computer vision techniques.

At the core of this system is the integration of physical token verification with real time vehicular identification. The architecture utilizes localized MFRC522 RFID readers positioned at outer and inner gates to initiate the scanning process, which subsequently triggers an Optical Character Recognition (OCR) pipeline powered by Python and EasyOCR. By cross referencing the registered identity of the scanned RFID tag against the physical license plate extracted from a live camera feed, the system ensures that the vehicle attempting to pass perfectly matches the presented credential.

A significant innovation of this project is the optimization of hardware resources through a dynamic, microcontroller driven surveillance mechanism. Instead of deploying multiple static cameras for entry and exit lanes, the system employs a servo actuated camera bracket that physically shifts its viewing angle (between 0° and 180°) based on which specific RFID reader is activated. This mechanical efficiency is paired with an intelligent software architecture that applies bilateral filtering, adaptive Gaussian thresholding, and a localized 1.5x digital zoom to stabilize and clean the captured frames, ensuring high OCR accuracy during hardware transitions.

Furthermore, the operational state of the facility is rigorously managed via a centralized SQLite database. This database not only maintains a permanent, timestamped log of all access attempts but also strictly enforces the anti-passback protocol by continuously updating and checking the binary (is_inside) status of every registered user. In the event of system failure or administrative necessity, the project features a secure 4x4 hardware matrix keypad that allows for an emergency manual override, locking the gate open while continuing to securely log the bypass activity to maintain institutional accountability. Ultimately, this thesis outlines the design, implementation, and evaluation of a seamlessly integrated cyber physical system aimed at modernizing parking infrastructure through cost effective, multi layered security. The high-level operational workflow of this proposed architecture is illustrated in Figure 1-1.

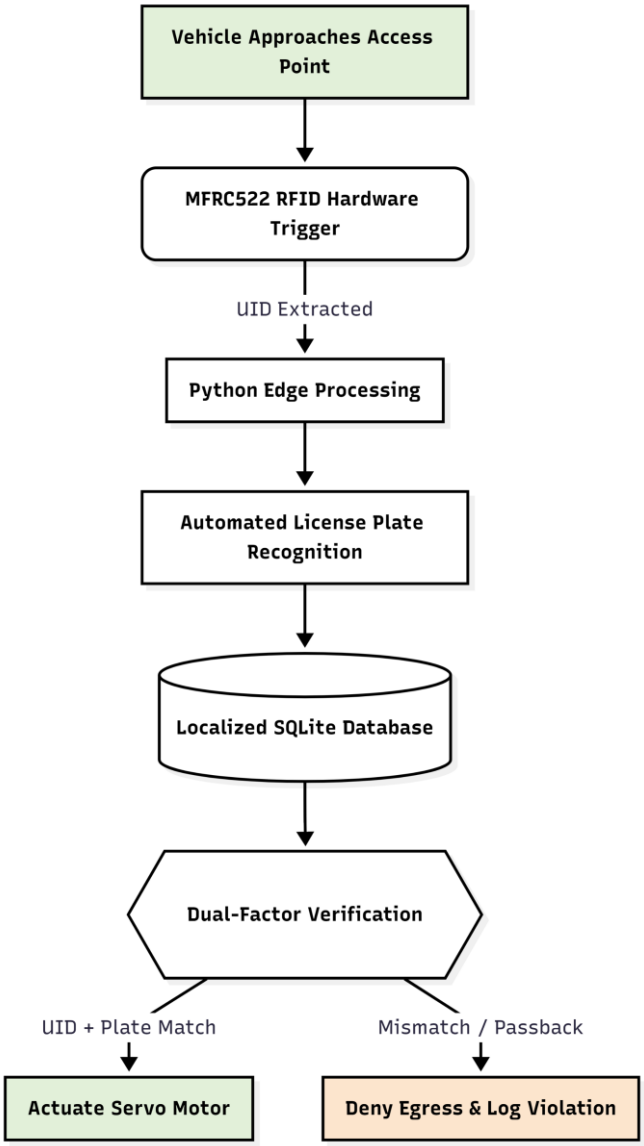


Figure 1-1 High level operational workflow of the proposed multi factor automated parking access control system

1.1 Background

The rapid expansion of urban centers, coupled with the increasing density of vehicular traffic in enclosed corporate and educational environments, has precipitated a critical need for intelligent parking management systems. As institutions transition toward "smart campus" and automated infrastructure frameworks, traditional access control architectures are becoming increasingly inefficient and insecure. Historically, parking administration has relied heavily on manual verification by security personnel or rudimentary, single factor electronic access methods. Among these, Radio Frequency Identification (RFID) technology emerged as the industry standard due to its rapid scanning capabilities, low implementation cost, and ease of deployment [1].

While RFID provides a swift clearance mechanism for approaching vehicles, it is fundamentally constrained by its authentication paradigm, the system authenticates the physical electronic token rather than verifying the actual identity of the vehicle or the driver. This structural limitation assumes that the possessor of the RFID card is the rightful, registered user, which leaves the parking infrastructure highly suspectible to credential sharing, physical theft, and logical spatial violations.

One of the most prevalent and critical security vulnerabilities in standard RFID based barrier systems is the "passback" exploit. This violation occurs when a registered user successfully gains entry to a restricted facility and subsequently passes their authorized token back to an unregistered driver waiting outside, thereby allowing multiple vehicles to enter using a single credential. Because standard legacy systems lack persistent spatial tracking (monitoring whether a credential is currently "inside" or "outside") and lack secondary physical identity verification, they cannot distinguish between the rightful owner of the card and an unauthorized possessor.

To counter these specific vulnerabilities, modern security frameworks have increasingly adopted computer vision technologies, particularly Automated License Plate Recognition (ALPR) powered by Optical Character Recognition (OCR) [3]. While ALPR systems resolve the identity verification issue by directly extracting and reading the physical vehicle's alphanumeric registration plate, they introduce significant logistical and financial barriers. Deploying a comprehensive ALPR architecture typically necessitates installing dedicated high resolution cameras, localized processing units, and independent network nodes for every single

entry and exit lane. This traditional multi camera redundancy exponentially increases both the initial hardware expenditure and the ongoing computational maintenance costs.

Consequently, there is a substantial engineering gap requiring the development of a cost effective, hybrid security framework. A system that unifies the rapid, localized triggering of MFRC522 RFID hardware with the rigorous identity verification of Python based OCR can establish a formidable dual factor authentication barrier. By automatically cross-referencing a digital credential's database record against physical vehicle data extracted in real time, unauthorized access can be systematically eliminated. Furthermore, addressing the hardware cost constraints requires innovative electromechanical solutions, such as utilizing a centralized, motorized dual position camera bracket capable of monitoring multiple lanes dynamically, thereby drastically reducing the need for redundant visual sensors.

1.2 Motivation

The fundamental drive behind this research stems from the pressing need to eradicate logical spatial violations, specifically the "passback" loophole, within conventional parking architectures. Traditional automated barrier gates operate on a flawed premise: they inherently trust the presented digital credential rather than verifying the physical presence of the authorized vehicle. This project is heavily motivated by the desire to enforce a strict identity correlation matrix mandating that the physical alphanumeric sequence of an approaching vehicle perfectly aligns with the registered string linked to a scanned MFRC522 RFID tag. By enforcing this dual factor cyber physical authentication protocol, the system aims to render stolen, cloned, or shared access cards entirely useless, thereby restoring high level security integrity to restricted campus and corporate environments.

Beyond pure security enhancements, a major economic motivation underpins this engineering endeavor. Commercial deployment of Automated License Plate Recognition (ALPR) is notoriously capital intensive, traditionally demanding a dedicated high definition optical sensor, localized processing node, and secure network uplink for every single ingress and egress point. This research is motivated by the engineering challenge of maximizing hardware optimization to reduce these prohibitive costs. We seek to demonstrate that comprehensive, multi lane surveillance can be achieved using a singular optical sensor mounted on a dynamic, servo actuated bracket. By engineering a system that mechanically shifts the camera's viewing angle between predefined spatial coordinates (0° for outer scans and 180° for inner scans) in

direct response to localized RFID reader triggers, the project aims to drastically reduce the necessary hardware footprint and democratize advanced security technologies.

Furthermore, the project is motivated by the inherent complexities of real world computer vision deployment. Extracting license plate data in variable environmental conditions frequently results in noisy or inaccurate datasets. Consequently, there is a strong motivation to engineer a resilient software pipeline that not only captures images but actively optimizes them. This drove the implementation of a localized 1.5x permanent digital zoom and the integration of OpenCV for real time frame stabilization, applying bilateral filtering and adaptive Gaussian thresholding before passing the data to the EasyOCR engine [10]. The ambition to achieve absolute data fidelity also motivated the creation of custom algorithmic filtering designed to automatically identify and discard irrelevant regional markers (such as "SINDH" or "PUNJAB"), ensuring only critical vehicular identifiers are processed.

Finally, the motivation extends beyond access control to encompass operational resilience and administrative accountability. A purely automated system risks causing severe logistical bottlenecks during hardware malfunctions, vehicular accidents, or network outages. Therefore, this project is motivated to integrate a secure, physical fail safe mechanism via a 4x4 hardware matrix keypad, enabling an encrypted emergency bypass sequence. Crucially, the motivation is not simply to provide a manual override, but to maintain absolute institutional transparency while doing so. To this end, all normal access grants, passback denials, and emergency keypad overrides are instantly and permanently archived within a centralized SQLite relational database. This ensures that the pursuit of advanced automated security does not compromise facility safety, rapid emergency egress, or strict administrative auditability.

1.3 Problem Statement

Contemporary access control infrastructures deployed in enclosed educational and corporate parking facilities predominantly depend on isolated, single factor authentication mechanisms, most commonly standalone Radio Frequency Identification (RFID) tokens. This singular dependency creates a critical security void, as the authentication process validates only the physical electronic token rather than verifying the true identity of the vehicle or the driver attempting entry. Consequently, these facilities are highly susceptible to security breaches such as credential sharing, token theft, and the passback exploit, wherein a single authorized token is sequentially reused to grant access to multiple unauthorized vehicles. Because standard legacy systems lack a persistent spatial tracking mechanism to determine whether a vehicle is

currently inside or outside the premises, they cannot logically prevent these spatial access violations. These specific structural vulnerabilities and logical exploits inherent in standard RFID access infrastructure are mapped out in Figure 1-2.

While the integration of computer vision, specifically Automated License Plate Recognition (ALPR), resolves the identity verification deficit by reading the vehicle's physical registration plate, it introduces severe logistical and financial barriers. Standard multi lane ALPR architectures necessitate highly redundant and expensive hardware deployments. They typically require independent high resolution optical sensors, localized processing nodes, and separate network uplinks for every individual ingress and egress point. This physical redundancy makes comprehensive, dual lane security financially prohibitive for many institutions, highlighting a significant engineering gap in resource optimization.

Furthermore, deploying ALPR algorithms in real world environments frequently yields noisy, distorted, or inaccurate data due to variable lighting, mechanical vibrations, and the presence of irrelevant regional text on license plates (e.g., "SINDH" or "PUNJAB"). Existing low cost systems often lack the localized image pre processing required to dynamically stabilize visual frames and filter out non essential alphanumeric characters without relying on heavy cloud computing resources. Additionally, strictly automated security systems pose a significant operational risk during hardware failures or emergencies, where rapid facility egress is mandated. Relying entirely on digital authentication without a secure, manually operated fail safe can lead to dangerous logistical bottlenecks.

Therefore, the core problem this project addresses is the absence of a cost effective, unified dual factor authentication framework capable of comprehensively eliminating spatial access violations without incurring prohibitive hardware overhead. This project must engineer an Automated Parking Access Control System (PACS) that firmly synchronizes physical MFRC522 RFID hardware triggers with Python driven EasyOCR visual verification. To resolve the hardware redundancy issue, the system must dynamically optimize resource utilization by employing a microcontroller actuated servo bracket to mechanically reposition a single optical sensor between the entry and exit lanes. Furthermore, it must strictly enforce a programmatic anti-passback protocol by persistently tracking vehicular states via a localized SQLite database and incorporate a fully logged, hardware based matrix keypad to securely manage and audit emergency bypass overrides.

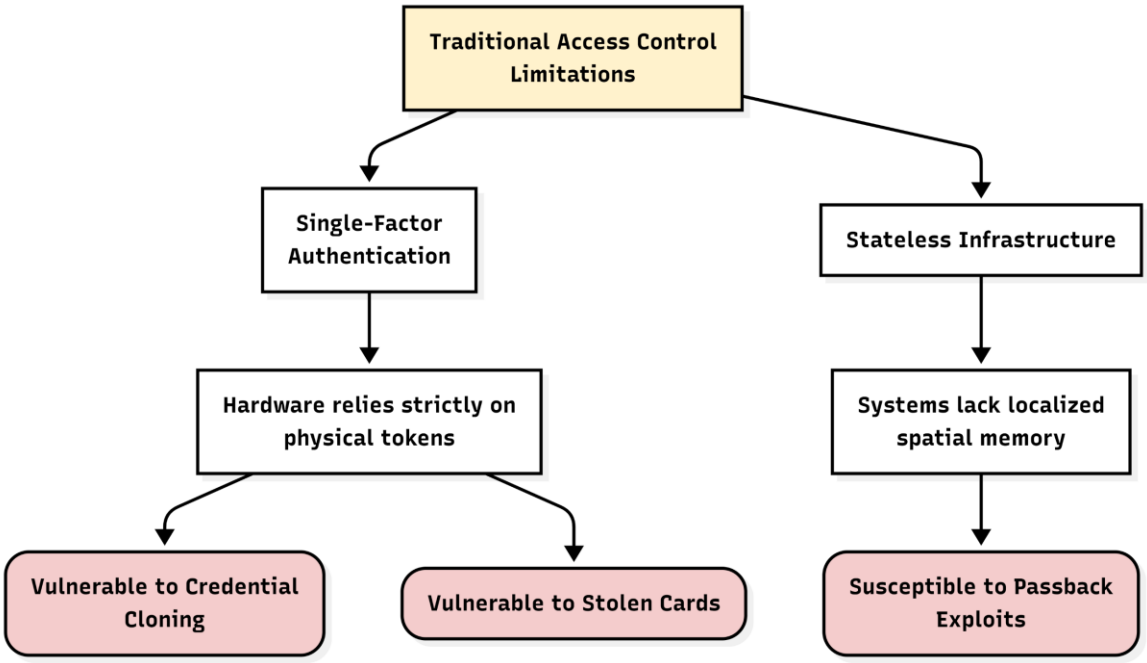


Figure 1-2 Specific structural vulnerabilities and logical exploits inherent in standard RFID access infrastructure

1.3.1 Complex Engineering Problem

The conceptualization, design, and implementation of this dual-factor automated parking framework inherently satisfy the criteria of a Complex Engineering Problem (CEP). The project necessitates the convergence of multidisciplinary engineering domains, specifically requiring the seamless integration of low-level embedded electromechanical control with high level computer vision algorithms and relational database management. Addressing the operational vulnerabilities of traditional access barriers while strictly adhering to hardware cost constraints demands advanced analytical reasoning and specialized engineering design. The specific attributes that qualify this system as a CEP are detailed below.

Table 1: Complex Engineering Problem

WPs	Description	Validation within the Project Scope
WP1	Depth of Knowledge Required	**WK4 (Specialist Knowledge):** Necessitates advanced expertise to interface high-level Python processing pipelines (utilizing OpenCV and EasyOCR) with low-level C++ embedded architecture via strictly timed asynchronous Serial communication. **WK5 (Engineering Design):** Requires the architectural design of a localized anti-passback state machine within an SQLite database, operating concurrently with the mechanical engineering of a dual-position servo-actuated optical mount **WK8 (Research Literature):** Demands comprehensive analysis of image processing literature to implement optimal noise-reduction algorithms—specifically bilateral filtering and adaptive Gaussian thresholding—to successfully extract alphanumeric data under variable physical conditions.
WP2	Range of conflicting requirements	The infrastructure mandates rigorous, independent visual verification for two spatially distinct access points (the outer entry lane and the inner exit lane). However, this operational requirement directly conflicts with the strict economic constraint of minimizing hardware deployment to ensure commercial viability. This conflict was successfully resolved by engineering a dynamic, microcontroller-actuated servo bracket capable of precisely shifting a singular optical sensor between a 0-degree (outer) and 180-degree (inner) viewing axis in direct response to localized MFRC522 RFID triggers.
WP3	Familiarity of issues	The integration of a dynamically rotating camera apparatus introduced a highly specific operational issue: severe optical motion blur and frame distortion resulting from the physical transition of the sensor. This unfamiliar issue was systematically resolved by engineering a software intervention pipeline that introduces a precise mechanical delay, flushes obsolete buffered frames from the OpenCV memory stream, and applies a permanent 1.5x localized digital zoom to secure a perfectly stabilized image prior to initiating the Optical Character Recognition (OCR) sequence.

1.4 Contributions

This research project delivers a fully integrated cyber physical architecture that significantly advances the operational security and resource efficiency of automated parking facilities. By successfully bridging low level embedded hardware control with high level machine vision and localized database management, the project introduces several novel engineering contributions to the domain of access control. The primary contributions of this thesis are detailed as follows:

- **Development of a Dual-Factor Cyber-Physical Authentication Framework:** The most prominent contribution is the engineering of a strictly synchronized, two tier verification pipeline. The system successfully decouples traditional single factor reliance by requiring an initial hardware trigger from localized MFRC522 Radio Frequency Identification (RFID) readers, which immediately initiates a secondary software driven optical verification process using Python and EasyOCR. This ensures that barrier actuation is strictly contingent upon a perfect match between the digital database identity and the physical vehicular license plate.
- **Engineering of a Dynamic Optical Repositioning Mechanism:** To resolve the economic constraints of deploying multiple high resolution cameras for separate ingress and egress lanes, a dynamic electromechanical viewing apparatus was developed. This contribution involves a microcontroller driven servo bracket that actively mechanically shifts a singular camera's focal trajectory between a 0-degree angle (for outer entry scans) and a 180-degree angle (for inner exit scans) in direct response to the specific RFID node activated on the shared SPI bus.
- **Implementation of a Strict Algorithmic Anti-Passback Protocol:** The project contributes a rigorous spatial tracking logic governed by a centralized SQLite relational database. By continuously updating and querying a binary (is_inside) state assigned to every authorized user's profile, the system mathematically eliminates logical access violations. The algorithm instantly identifies and denies any entry attempt by a credential that is already logged as being inside the facility, or conversely, denies exit to a credential not recognized as currently present.
- **Design of a Robust Vision Pre-Processing Pipeline:** Addressing the inherent instability caused by mechanical camera repositioning, the project contributes a sophisticated image stabilization and noise-reduction sequence. This software pipeline

actively flushes obsolete buffered frames and applies a permanent 1.5x localized digital zoom to secure a centralized region of interest. Furthermore, it employs bilateral filtering, adaptive Gaussian thresholding, and a customized alphanumeric blacklist (filtering out regional identifiers like "SINDH" or "PUNJAB") to guarantee high fidelity data extraction prior to executing the OCR engine.

- **Integration of an Auditable Emergency Override System:** To ensure logistical safety without compromising institutional accountability, a hardware based fail safe mechanism was natively integrated into the system's core loop. This contribution utilizes a 4x4 matrix keypad allowing administrators to trigger an encrypted bypass sequence, locking the gate open during emergencies. Crucially, the Python backend intercepts these manual override signals and permanently archives them within the access_logs database schema, ensuring every manual bypass is strictly timestamped and audited.

1.4.1 Relation with the Sustainable Development Goals

The integration of advanced cyber physical systems into urban and institutional environments plays a pivotal role in advancing global sustainability. This dual factor automated parking access control framework directly aligns with the United Nations Sustainable Development Goals (SDGs) by fostering technological innovation, enhancing infrastructural safety, and optimizing resource management within modern smart cities.

Table 2: SDGs table

SDG No	Description of SDG	Target	Relevance to our Project
SDG 8	Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.	Achieve higher levels of economic productivity through diversification, technological upgrading and innovation, including through a focus on high value added and labour intensive sectors.	By fully automating the vehicular access control process through integrated Optical Character Recognition (OCR) and RFID scanning, this project reduces the reliance on manual security personnel for repetitive logging tasks. This allows human resources to be reallocated toward higher level security management and facility oversight. Furthermore, the cost effective deployment of a dynamic single camera system provides an economically viable technological upgrade for institutions, fostering sustainable economic innovation without prohibitive financial burdens.
SDG 9	Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.	By 2030, upgrade infrastructure and retrofit industries to make them sustainable, with increased resource use efficiency and greater adoption of clean and environmentally sound technologies and industrial processes.	This cyber-physical parking framework innovates facility infrastructure by maximizing electronic resource efficiency. By engineering a motorized single camera mechanism to monitor multiple access lanes, the system drastically reduces hardware redundancy, lowering both electronic waste and initial deployment costs. The seamless integration of lightweight Python machine vision with embedded Arduino microcontrollers demonstrates a highly sustainable, scalable engineering design suitable for resilient institutional infrastructure.

SDG 11	Make cities and human settlements inclusive, safe, resilient and sustainable.	By 2030, provide access to safe, affordable, accessible and sustainable transport systems for all, improving road safety, notably by expanding public transport and smart infrastructure networks.	The development of this dual factor automated parking system directly contributes to the creation of smart, secure urban settlements. By strictly eliminating passback violations and unauthorized vehicular entry through rigorous database driven ALPR and RFID cross verification, the system significantly enhances the localized safety of urban parking infrastructure. Additionally, expediting the authentication process at the barrier minimizes vehicular idling, contributing to smoother traffic flow and marginally reducing localized emissions within crowded campus environments.

2

Literature Review

This chapter reviews the foundational technologies and existing research relevant to automated parking access control systems. It examines traditional RFID implementations, advancements in Automated License Plate Recognition (ALPR), and the persistent infrastructural vulnerabilities of logical spatial violations, specifically the passback exploit. Furthermore, it explores modern engineering strategies for optimizing optical hardware deployment and integrating secure physical fail safes within cyber physical environments.

2.1 Traditional Access Control & RFID Implementation

Historically, vehicular access control architectures have heavily depended on manual verification processes, which were susceptible to human error, high labor costs, and significant logistical bottlenecks during peak traffic hours. To automate these environments, the security industry universally adopted Radio Frequency Identification (RFID) as the foundational technology for frictionless access control. Academic and industrial literature establishes RFID as a highly efficient mechanism due to its rapid, contactless data transmission capabilities and overall economic viability for large scale deployments [1]. In practical architectural implementations, systems utilizing embedded readers such as the MFRC522 modules integrated via the SPI bus in this project operate by emitting localized electromagnetic radio frequencies [19]. These frequencies energize passive, battery-less tags carried by users, enabling a cryptographic handshake that extracts a pre-programmed Unique Identification (UID) payload to rapidly process a gate entry.

However, a strong consensus within contemporary cybersecurity and access control literature highlights a severe structural flaw in standalone RFID infrastructur, it operates strictly as a single factor authentication mechanism. Standard barrier systems rely entirely on verifying the presence of the digital token itself. The underlying algorithm implicitly, and often incorrectly,

assumes that the physical possessor of the token is the authorized registered user linked to that credential in the database. Consequently, legacy RFID architectures remain inherently vulnerable to an array of physical and logical security breaches. Because passive RFID tags transmit their UID openly when energized by a compatible reader, they are highly susceptible to credential cloning using low cost hardware [2]. Furthermore, physical card theft and unauthorized token sharing among peers remain critical blind spots for these traditional systems [2].

Beyond credential spoofing, standalone RFID frameworks suffer from a lack of inherent spatial statefulness. Traditional embedded microcontrollers simply read a card and trigger a relay, possessing no awareness of the user's actual physical location relative to the facility's perimeter. This absence of localized memory and relational tracking creates the environment for the "passback" exploit, where a single card can be scanned sequentially to allow a convoy of unauthorized vehicles through a perimeter. Solving this requires transitioning from a stateless hardware trigger to a stateful, database driven tracking environment.

To comprehensively counteract these vulnerabilities, modern engineering research strongly advocates for the integration of multi tiered, cyber physical authentication frameworks that mandate a secondary layer of physical identity verification. The parking system engineered in this thesis directly aligns with and advances these modern security paradigms. Instead of allowing the RFID scan to directly actuate the barrier, this project relegates the dual MFRC522 readers to function strictly as high speed, asynchronous hardware triggers. When a card is presented, the embedded microcontroller merely captures the UID and transmits it via serial communication to a centralized Python backend. This architectural shift ensures that the localized RFID hardware does not possess the authority to grant access, rather, it serves to initiate a stringent secondary machine vision pipeline and query a relational database, guaranteeing that the vehicle is visually verified and spatially authorized before the barrier servo is engaged. A direct architectural comparison between traditional single-factor RFID systems and the proposed dual-factor cyber-physical verification workflow is detailed in Figure 2-1.

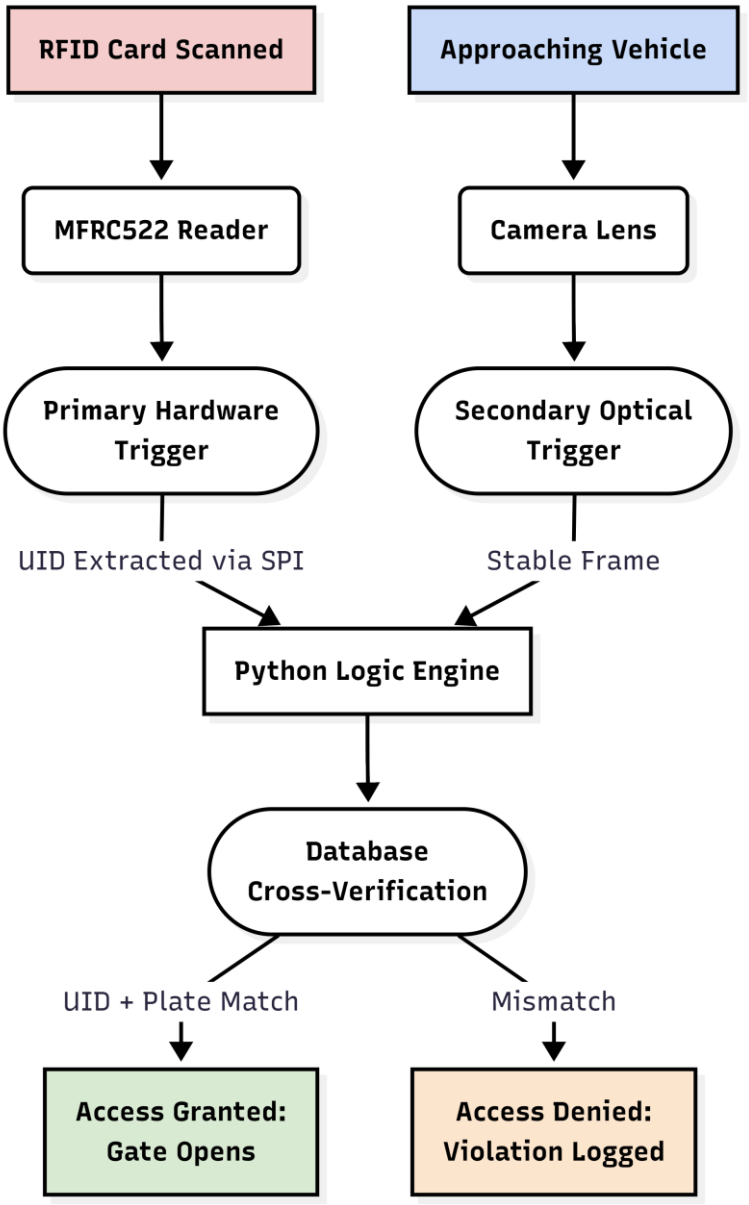


Figure 2-1 Architectural comparison between traditional single factor RFID systems & the proposed dual factor cyber physical verification workflow

2.2 Advancement in Automated License Plate Recognition (ALPR) & Computer Vision

To resolve the inherent identity verification vulnerabilities associated with isolated RFID networks, the access control and security industries have increasingly integrated Automated License Plate Recognition (ALPR) systems. ALPR bridges the gap between digital credentials and physical presence by utilizing Optical Character Recognition (OCR) to extract and verify the actual alphanumeric registration of an approaching vehicle. Traditional OCR engines historically relied heavily on rigid algorithmic pattern matching, edge detection, and dimensional heuristics. While effective in highly controlled environments, these legacy systems frequently struggled with the variable fonts, regional markers, non-standard plate dimensions, and environmental noise typical of real world vehicular traffic [3].

Recent computational literature demonstrates a paradigm shift toward deep learning based OCR frameworks, which significantly outperform legacy software in both confidence intervals and overall character accuracy. Modern extraction engines that leverage sophisticated neural networks, such as the EasyOCR framework utilized in this project, have been shown to interpret complex spatial boundaries and distorted text much more effectively [10], [12]. Unlike older heuristic models, these neural architectures can dynamically adapt to varying light conditions and skewed perspectives, yielding substantially lower error rates in challenging visual environments.

Despite these algorithmic advancements, capturing highly stable optical data from moving vehicles or, in the case of this project, from a dynamically rotating mechanical camera bracket remains a significant physical challenge. Academic reviews of machine vision deployments heavily emphasize that the raw accuracy of a neural network is directly bottlenecked by the quality of the optical input. Therefore, implementing a robust, localized image pre-processing pipeline is critical. The system developed in this thesis synthesizes this literature by deploying an EasyOCR framework that is heavily fortified with custom OpenCV (Computer Vision) pre-processing.

To guarantee maximum data fidelity prior to character extraction, the Python architecture intercepts the raw camera feed and applies a sequence of aggressive transformations. First, the system counteracts mechanical optical distortion and frame shifting by employing a localized, permanent 1.5x digital zoom, which actively centralizes and stabilizes the region of interest

before processing begins. Subsequently, the system converts the centralized frame to grayscale and applies a bilateral filter; this specific algorithmic choice is crucial as it heavily reduces background noise while preserving the sharp edges of the alphanumeric characters. Following noise reduction, an adaptive Gaussian thresholding technique is applied to sharply binarize the image, effectively isolating the dark text of the license plate from its reflective background regardless of ambient lighting conditions [4]. This sequential OpenCV image pre-processing pipeline, applied prior to localized EasyOCR character extraction, is visualized in Figure 2-2.

Finally, ALPR literature often notes the difficulty of parsing extraneous visual data, such as regional text or governmental slogans printed on standard license plates, which can confuse the verification logic. This project addresses this data cleaning challenge through localized post processing. Before the extracted text is verified against the SQLite database, a custom algorithmic filter strips all non-alphanumeric characters and checks the string against a predefined blacklist of regional identifiers (e.g. "SINDH", "PUNJAB", "PAKISTAN", "GOVT"). This ensures that only the critical, identifying vehicle registration sequence is processed, resulting in a highly optimized, resilient, and accurate dual factor authentication pipeline.

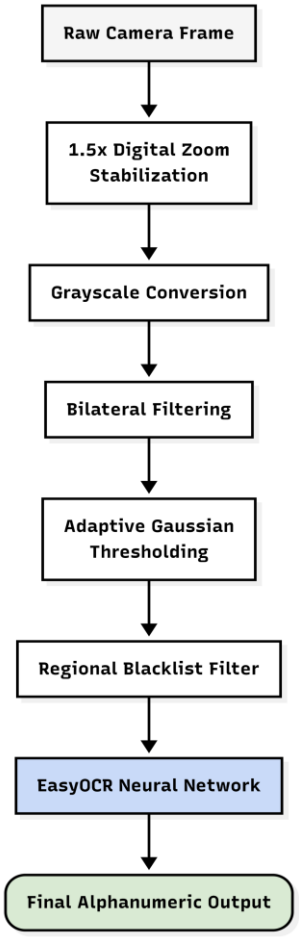


Figure 2-2 The sequential OpenCV image pre-processing pipeline applied prior to localized EasyOCR character extraction

2.3 Addressing the Passback Security Vulnerability & Spatial Tracking

In the domain of physical security and automated access control, one of the most widely documented logical exploits is the "passback" violation. This infrastructural vulnerability manifests when a registered credential holder successfully authenticates at an entry point and subsequently physically transfers their token to an unauthorized individual waiting outside the perimeter. Because standard, stateless Radio Frequency Identification (RFID) configurations strictly verify the digital signature of the card rather than maintaining a logical spatial sequence, they inadvertently permit multiple distinct entries using a single digital identity. Enterprise-grade security literature heavily mandates the implementation of "anti passback" mechanisms to enforce rigorous spatial tracking. However, standard commercial solutions typically rely on complex, centralized server infrastructures and continuous network uplinks. These traditional architectures are often economically unfeasible and technically over provisioned for smaller scale educational or corporate deployments, highlighting a need for decentralized, localized solutions.

This project resolves this specific literature gap by engineering a highly efficient, localized anti passback protocol built directly upon a lightweight SQLite relational database architecture. Instead of relying on a stateless hardware trigger that blindly actuates a barrier, the system programmatically tracks vehicular presence by assigning a dynamic binary state variable, designated as is_inside, to every authorized user profile within the authorized_users data table. This approach leverages fundamental relational database design to maintain absolute statefulness without requiring external cloud connectivity.

The Python backend operates as a continuous state machine that actively monitors and enforces these spatial rules. When an asynchronous signal is received from the outer MFRC522 reader for a specific unique identifier (UID), the script queries the relational database before initiating the vision pipeline. If the database query reveals that the user is logically already inside the facility (is_inside = 1), the system intelligently rejects the credential, identifying it as a spatial continuity violation. The software then intercepts this event and permanently archives a "DENIED_PASSBACK_ENTRY" violation within the access_logs table for administrative auditing.

Conversely, if an inner reader scan occurs for a vehicle that the database registers as currently being outside (is_inside = 0), the system denies the egress attempt and generates a "DENIED_PASSBACK_EXIT" error. By seamlessly integrating asynchronous hardware signaling with robust, localized relational database management, this methodology successfully establishes enterprise grade spatial security within a highly cost effective computational framework, completely neutralizing the passback exploit. The complete logical flow diagram of this spatial anti-passback protocol, managed by the localized SQLite database architecture, is illustrated in Figure 2-3.

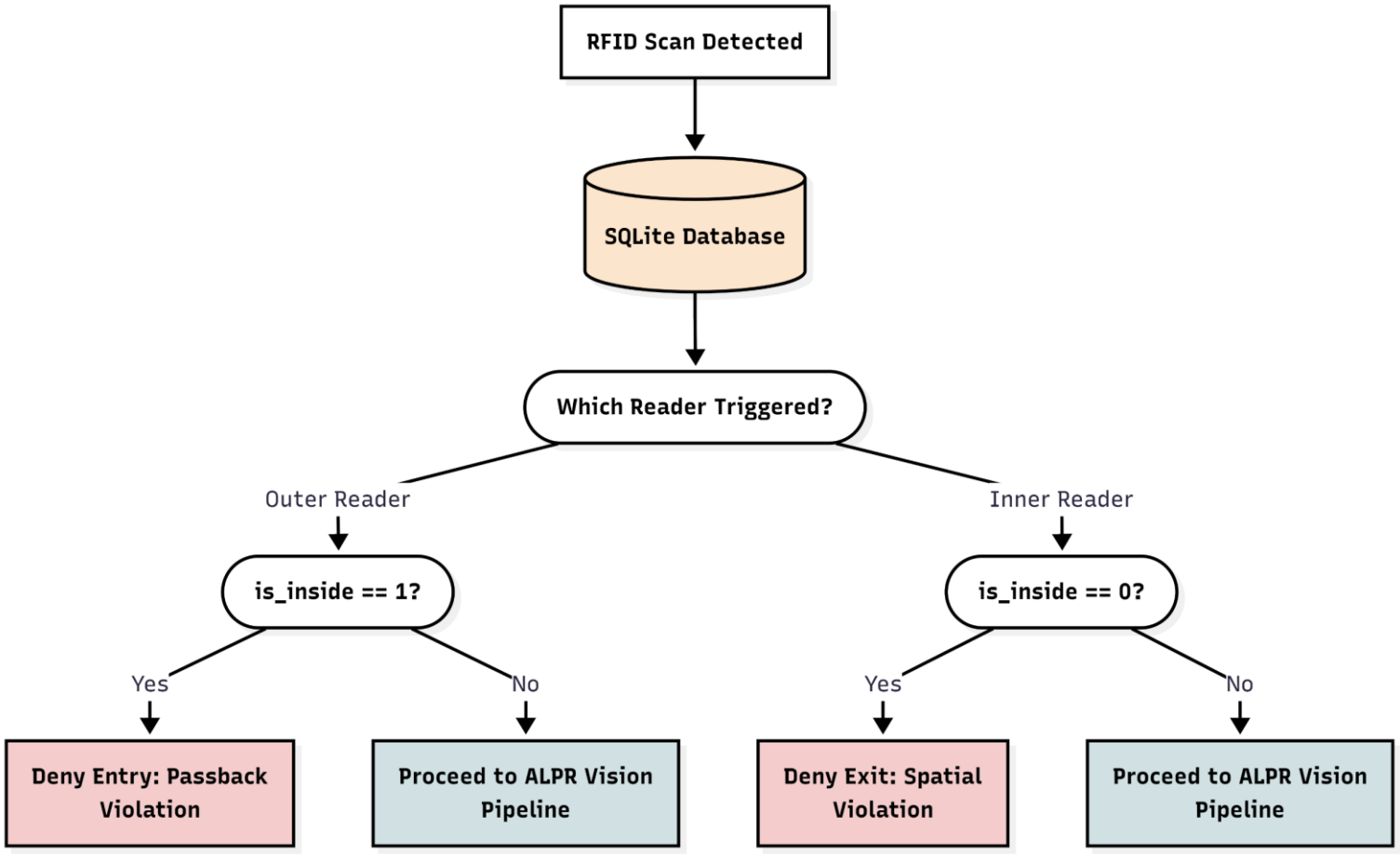


Figure 2-3 Logical flow diagram of the spatial anti passback protocol managed by the localized SQLite database architecture

2.4 Resource Optimization via Dynamic Sensor Positioning

While the integration of multi-factor authentication combining Radio Frequency Identification (RFID) and Automated License Plate Recognition (ALPR) guarantees exceptional operational security, academic reviews of physical infrastructure deployments frequently highlight a critical economic hurdle hardware redundancy. Standard ALPR implementations require a dedicated optical sensor (camera), localized processing hardware, and independent network routing for every individual traffic lane. In a typical two way access point (comprising one entry lane and one exit lane), traditional security architectures demand two separate camera setups operating continuously. This linear increase in hardware expenditure significantly escalates capital expenditure (CapEx) and operational maintenance overhead, limiting the deployment of advanced cyber physical security systems in cost sensitive institutional environments.

In computer vision and robotics literature, active vision systems which employ electromechanical actuators to dynamically orient optical sensors toward specific regions of interest are widely recognized for maximizing sensing coverage while minimizing sensor count. However, there is a distinct scarcity of research applying dynamic optical repositioning specifically to multi lane vehicular access control. Most commercial architectures opt for static camera arrays due to the technical challenges associated with physical sensor movement, such as mechanical wear, optical settling time, and frame blurring during spatial repositioning.

This project directly addresses this literature gap by introducing a dynamic electromechanical viewing mechanism that optimizes hardware utilization. Rather than deploying redundant cameras, the system utilizes a single optical camera mounted on a servo actuated bracket managed by an Arduino microcontroller. The camera's physical viewing axis is dynamically manipulated across a 180-degree spectrum based on asynchronous signals originating from the RFID readers on the shared SPI bus. When an incoming vehicle triggers the outer RFID reader, the microcontroller actuates the camera servo to rotate to 0° (camOuterAngle). Conversely, when an egressing vehicle triggers the inner RFID reader, the servo smoothly rotates the camera to 180°. The operational workflow of this dynamic camera bracket positioning system is outlined in Figure 2-4.

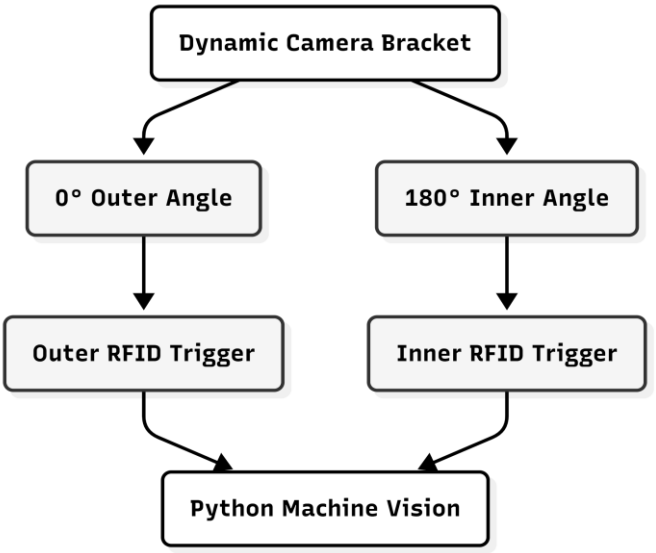


Figure 2-4 Operational workflow of the dynamic camera bracket

To overcome the inherent physical challenges of mechanical camera repositioning specifically optical instability and motion blur the software and hardware pipelines are engineered with dedicated synchronization protocols. The embedded C++ firmware introduces an explicit mechanical settling delay (500ms) immediately after driving the camera servo to allow physical vibrations to dissipate. On the host machine, the Python processing backend implements a 2-second delay followed by a frame buffer flushing sequence (for _ in range(5): cap.read()). This step purges obsolete, motion blurred frames accumulated in OpenCV's frame pipeline during the physical rotation, capturing a perfectly stable optical image. Furthermore, a localized 1.5x digital zoom (apply_zoom) is applied to crop and re-center the stable frame, isolating the license plate zone without requiring additional physical lenses. By synchronizing electromechanical positioning with targeted image pre processing, the architecture effectively reduces required optical hardware by 50% while maintaining high OCR recognition accuracy.

2.5 Emergency Protocol & Administrative Auditing

A critical concern repeatedly raised in academic literature regarding fully automated cyber physical infrastructure is system resilience during emergency scenarios or unexpected hardware degradation. Access control systems that rely exclusively on complex digital authentication layers such as machine vision or database querying can inadvertently create severe logistical bottlenecks if network connectivity fails, sensor occlusion occurs, or rapid facility egress is mandated. Infrastructural safety standards and industrial best practices dictate the mandatory inclusion of secure, manual fail safes. However, a persistent challenge in deploying manual overrides is that they frequently bypass the digital architecture entirely, leaving no administrative audit trail and thereby creating a significant security blind spot.

This project natively incorporates these crucial safety paradigms by engineering a fail safe mechanism that seamlessly integrates with the system's digital auditing pipeline. To ensure rapid administrative intervention without compromising security, a dedicated 4x4 matrix keypad is wired directly into the primary Arduino microcontroller's execution loop. This localized hardware component is programmed with a secure emergency bypass sequence. When the correct passcode is authenticated on the edge device, the microcontroller immediately forces the barrier servo into a sustained open position, effectively overriding the standard automated closure timer.

Crucially, to maintain absolute institutional transparency, this physical override does not circumvent the software's tracking architecture. Upon activation, the embedded firmware actively transmits an override state signal (such as EMERGENCY_OVERRIDE_ON or EMERGENCY_OVERRIDE_OFF) via the asynchronous serial interface to the Python backend. The primary computational script intercepts these manual override events and permanently commits them into the SQLite database's access_logs table. By recording a predefined identifier of "KEYPAD_BYPASS" alongside the precise timestamp and operational status (e.g., "EMERGENCY_OPEN" or "EMERGENCY_CLOSE"), the system guarantees that every deployment of the emergency protocol remains fully accountable and chronologically auditable. This methodology successfully resolves the literature gap between necessary physical safety mechanisms and rigorous administrative oversight. The state transitions and logical execution of this physical emergency protocol are mapped out in Figure 2-5.

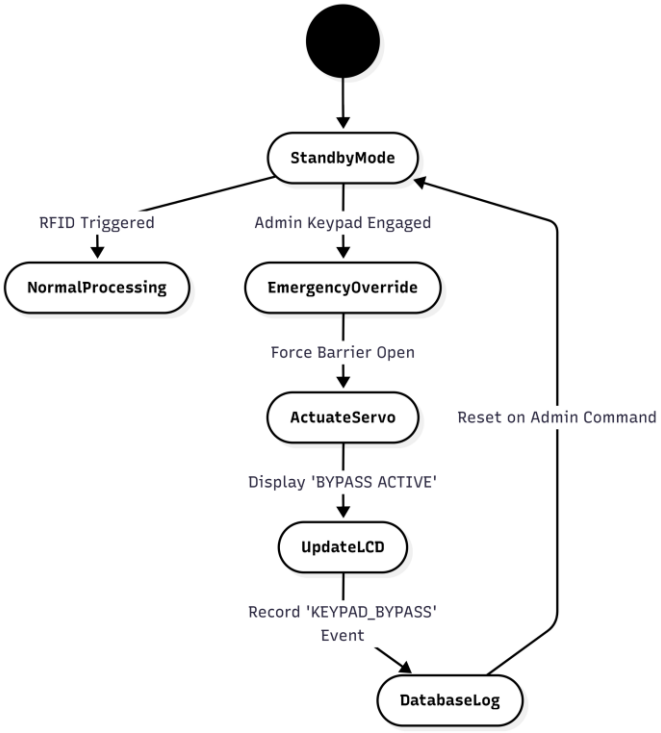


Figure 2-5 Diagram detailing the physical emergency protocol

2.6 Relational Database in Edge Security Architecture

In the evolution of cyber physical access control infrastructures, data storage and retrieval mechanisms have undergone a significant paradigm shift from centralized cloud architectures toward localized edge computing. Academic literature concerning smart infrastructure frequently emphasizes the operational vulnerabilities associated with cloud-dependent databases, specifically regarding network latency, bandwidth consumption, and external connectivity failures. When automated barriers rely on remote server validation to process entry credentials, even transient network drops can induce severe logistical bottlenecks and safety hazards at the facility perimeter. To guarantee high availability, low latency decision-making, and robust data privacy, contemporary engineering frameworks heavily advocate for the deployment of lightweight, serverless relational databases positioned directly at the network edge.

This project comprehensively adopts this edge computing philosophy by embedding a localized SQLite relational database architecture (parking_system.db) directly within the host machine's processing environment [8]. By utilizing a self contained, file-based database engine

rather than a heavy, continuously running background SQL server, the system ensures instantaneous read and write execution. This methodology drastically reduces the computational overhead on the local machine while immunizing the security framework against internet outages.

Furthermore, literature on database management within security applications stresses the importance of relational structuring for accurate audit trails. The system developed in this thesis implements a strict relational schema comprising two primary tables, authorized_users for identity and spatial tracking, and access_logs for chronological auditing. This localized, relational architecture guarantees that the critical querying of a user's is_inside binary state, the validation of alphanumeric license plates, and the subsequent logging of timestamped entry events occur with near zero computational latency. Consequently, the physical barrier actuates in true real time, delivering enterprise grade data integrity and operational resilience without the financial or technical dependencies of external cloud hosting.

2.7 Asynchronous Cyber Physical Communication

In the design of modern cyber physical systems, bridging the gap between high level computational software and low level embedded hardware presents a complex engineering challenge. Academic literature regarding embedded systems frequently highlights the inherent synchronization issues that arise when interfacing resource heavy, operating system dependent languages (such as Python) with real time, resource constrained microcontrollers (such as Arduino) [14], [15]. When high level machine vision pipelines attempt to communicate directly with localized hardware, poorly synchronized data transmission can lead to serial buffer overflows, blocking operations, and fatal system desynchronization. To ensure operational stability, literature advocates for the implementation of robust, asynchronous serial communication protocols that allow both systems to operate independently without freezing or stalling waiting for the other's response.

This project successfully implements a highly synchronized asynchronous communication architecture to bridge the Python machine vision backend with the localized Arduino hardware. Communication is established over a localized USB serial interface operating at a standardized baud rate, ensuring reliable, low latency data packets. To prevent the Arduino from stalling its primary execution loop while waiting for the computationally heavy Python OCR script to finish processing, the embedded C++ firmware relies on non-blocking programming paradigms. Instead of using restrictive delay() functions that freeze the microcontroller, the

firmware utilizes the millis() function to asynchronously manage the gate's open timer state [16], allowing the hardware to continuously monitor the physical keypad and RFID sensors while waiting for digital feedback.

Furthermore, the data payload structures are strictly optimized to prevent buffer saturation. When an RFID tag is scanned or a keypad override is engaged, the Arduino transmits brief, highly specific string identifiers (e.g., "OUTER_UID" or "EMERGENCY_OVERRIDE_ON") rather than continuous data streams. The Python backend operates in a continuous polling loop, explicitly checking for waiting bytes before initiating a read sequence. Upon successfully authenticating a license plate against the database, the Python script replies with a singular byte character 'G' for Granted or 'D' for Denied. This minimalist payload guarantees instant hardware actuation. Finally, to ensure the system does not process stale or delayed hardware triggers a critical flaw noted in poorly designed cyber physical frameworks the Python architecture actively flushes the hardware input buffers (arduino.reset_input_buffer()) after every transaction, ensuring the communication pipeline remains clean and synchronized for the next vehicular event.

2.8 Human Machine Interfaces (HMI) in Access Control

In fully automated cyber physical environments, the absence of human security personnel necessitates the implementation of highly effective Human Machine Interfaces (HMI). Academic literature focusing on user experience and infrastructural logistics heavily emphasizes the psychological and operational importance of providing immediate, localized visual feedback to end users. When an automated barrier lacks a clear communication channel, drivers are often left unaware of the system's internal processing state. This lack of transparency frequently leads to user confusion, repetitive and redundant credential scanning, and subsequent vehicular bottlenecks at critical ingress and egress chokepoints. Modern engineering standards dictate that a successful access control deployment must bridge the cognitive gap between the complex backend processing architecture and the human operator through instantaneous visual cues.

This project directly incorporates these HMI best practices by embedding an I2C based Liquid Crystal Display (LCD) directly into the primary Arduino microcontroller circuitry. Rather than

operating as a silent barrier, the system's firmware is explicitly programmed to act as a dynamic communication matrix, providing real-time state updates to the driver.

By tightly coupling the visual output with the asynchronous hardware and software pipelines, the localized HMI module accurately reflects the exact status of the authentication process. When an RFID token is initially presented, the microcontroller immediately updates the display to read "SCANNING", reassuring the user that the hardware trigger has been successfully registered and the backend optical verification sequence has commenced. Following the Python driven OCR processing, the system provides definitive, actionable feedback. If the alphanumeric license plate matches the database credential, the system outputs "ACCESS GRANTED". Conversely, if a spatial passback violation or a plate mismatch occurs, the display explicitly notifies the user with an "ACCESS DENIED" and "CHECK PLATE/UID" prompt.

Furthermore, the HMI architecture is deeply integrated with the system's emergency protocols. If the matrix keypad is utilized to manually override the barrier, the display immediately shifts to "EMERGENCY MODE" and "BYPASS ACTIVE". This continuous, state aware visual feedback minimizes cognitive friction at the facility perimeter, heavily reduces vehicular idling times, and ensures smooth, orderly throughput even during complex multi factor authentication sequences.

3

System Architecture & Methodology

This chapter strictly details the practical engineering methodology, hardware architecture, and software configurations utilized to construct the proposed multi factor parking access control system. Transitioning from the theoretical frameworks established in the literature review, this chapter provides a comprehensive, component level blueprint of the physical edge devices, the sequential machine vision pipelines, and the highly synchronized communication protocols. The system is fundamentally divided into two synchronized domains, the localized embedded hardware tier and the computationally heavy, object oriented Python backend.

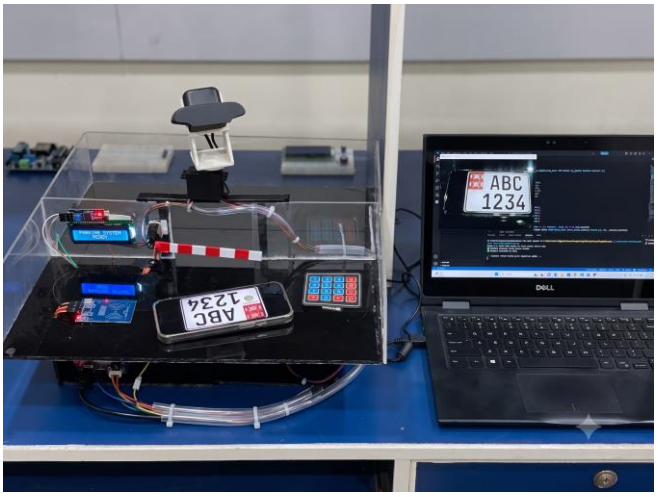


Figure 3-1 Computer Vision Based Parking Authorization & Authentication System Hardware & Software

3.1 Embedded Hardware Architecture & Edge Interfacing

The physical perimeter of the access control system is governed by a centralized microcontroller, which acts as the primary localized edge processing unit. In the context of cyber physical security, the edge layer is strictly responsible for managing all localized environmental triggers, actuating physical barriers, and providing real time cognitive feedback to the user. By isolating these hardware tasks on a dedicated microcontroller, the system avoids relying on the host computer's operating system for micro timing and hardware interrupts, thereby guaranteeing deterministic execution and immediate physical responsiveness.

The hardware schema integrates multiple complex subsystems, each utilizing distinct communication protocols to interface with the core microcontroller:

3.1.1 Dual RFID Subsystem Integration (SPI Protocol)

The primary authentication trigger relies on two distinct MFRC522 Radio Frequency Identification (RFID) modules one physically positioned at the entry lane to monitor incoming traffic, and the other at the exit lane to monitor egressing traffic. These modules operate at a standard high frequency band of 13.56 MHz, generating a localized electromagnetic field to briefly power passive user credentials.

To interface both readers simultaneously without saturating the microcontroller's limited digital input/output (I/O) pins, the architecture utilizes the Serial Peripheral Interface (SPI) bus. SPI is a synchronous serial communication protocol that allows multiple peripheral devices to communicate with a master controller using a shared data bus. Both MFRC522 readers share the Master Out Slave In (MOSI), Master In Slave Out (MISO), and Serial Clock (SCK) lines. To distinguish between an outer lane scan and an inner lane scan, the microcontroller assigns a unique Slave Select (SS) pin to each reader. When the microcontroller sequentially polls the readers, it pulls the respective SS line low, opening a dedicated communication channel to extract the 32-bit Unique Identification (UID) payload at high speeds [19].



Figure 3-2 Dual RFID subsystem configuration

3.1.2 Electromechanical Camera Actuation (PWM)

Instead of relying on a redundant, multi-camera array, the physical architecture optimizes resource utilization by mounting a single optical sensor onto a dynamic mechanical bracket. This bracket is driven by a standard micro servo motor. The microcontroller manages this electromechanical positioning using strictly calibrated Pulse Width Modulation (PWM) signals [17].

By varying the duty cycle of the digital square wave sent to the servo's control wire, the microcontroller precisely dictates the physical rotation of the bracket. A specific PWM pulse width corresponds to the 0° position (facing the outer RFID trigger), while an inverted pulse width corresponds to the 180° position (facing the inner RFID trigger). The hardware architecture inherently accounts for the physical inertia of the motor; a strict physical settling delay is programmed into the microcontroller's execution sequence immediately following a PWM state change to ensure the optical sensor ceases vibrating before the software tier captures the frame.



Figure 3-3 Electromechanical Camera Actuation

3.1.3 Human Machine Interface via I2C Bus

To maintain user compliance and reduce cognitive friction at the physical barrier, the edge architecture integrates a 16x2 Liquid Crystal Display (LCD). Standard LCD modules typically require up to six separate digital data pins to operate, which would severely restrict the microcontroller's remaining I/O availability. To circumvent this hardware limitation, the LCD is paired with an Inter Integrated Circuit (I2C) expansion backpack [18].

The I2C protocol reduces the physical wiring requirement to merely two communication lines, Serial Data (SDA) and Serial Clock (SCL). Through this bus, the microcontroller transmits hexadecimal addresses and string payloads to dynamically update the physical screen. This localized Human Machine Interface (HMI) operates entirely independently of the host machine's graphical user interface, instantly reflecting hardware states such as "SCANNING", "ACCESS GRANTED", or "EMERGENCY MODE".



Figure 3-4 Implementation of I2C LCD

3.1.4 Fail Safe Override Matrix (Digital Multiplexing)

To ensure compliance with physical safety regulations, a localized manual override system is engineered directly into the hardware tier using a 4x4 matrix keypad [20]. This matrix allows administrative personnel to input a localized bypass passcode during critical network failures or hardware malfunctions.

Because a standard 16 button array would theoretically require 16 separate input pins, the hardware utilizes a multiplexing matrix design, reducing the requirement to just 4 row pins and 4 column pins. The microcontroller executes a continuous localized polling loop, sequentially driving each row pin to a HIGH state while reading the capacitive state of the columns. If a connection is bridged by a physical keypress, the embedded firmware instantly registers the specific coordinate. To prevent false positive reads caused by the mechanical bouncing of the button contacts, the microcontroller implements a rigorous software-based debounce algorithm, guaranteeing that a single physical press transmits only a single localized command to the system's state machine.



Figure 3-5 Emergency Bypass System Using 4x4 Keypad

3.2 Computational Software Tier & Asynchronous Execution

While the microcontroller governs the physical edge boundaries and hardware interrupts, the complex logical routing, relational data validation, and optical character extraction are strictly offloaded to a high performance Python processing backend. Operating on the host machine, this computational software tier acts as the primary logical engine for the dual factor authentication system. To ensure absolute modularity, scalability, and maintainability across a complex cyber physical system, the software architecture is engineered using rigorous Object Oriented Programming (OOP) paradigms and asynchronous concurrency models.

3.2.1 Object Oriented System Encapsulation

In traditional procedural programming, maintaining state across continuous hardware loops, database connections, and real time computer vision frames frequently leads to memory leaks and variable collision. To circumvent this, the software tier encapsulates each major operational domain into dedicated class structures.

The architecture is broadly divided into several highly cohesive modules:

- **The Serial Bridge Class:** Responsible solely for managing the serial I/O buffers, establishing the COM port connection, and continuously polling for incoming byte payloads from the microcontroller.
- **The Database Manager Class:** Encapsulates the SQLite connection strings, cursor executions, and SQL query formulations. This class isolates the is_inside validation logic from the rest of the application, ensuring that database commits are atomic and secure.
- **The Vision Processor Class:** Manages the OpenCV frame buffer and the EasyOCR neural network. It actively maintains the centralized video stream and executes the image filtering pipeline upon command.
- **The System State Class:** Tracks system health, runtime metrics, and simulated edge power status, ensuring that software variables accurately reflect physical realities.

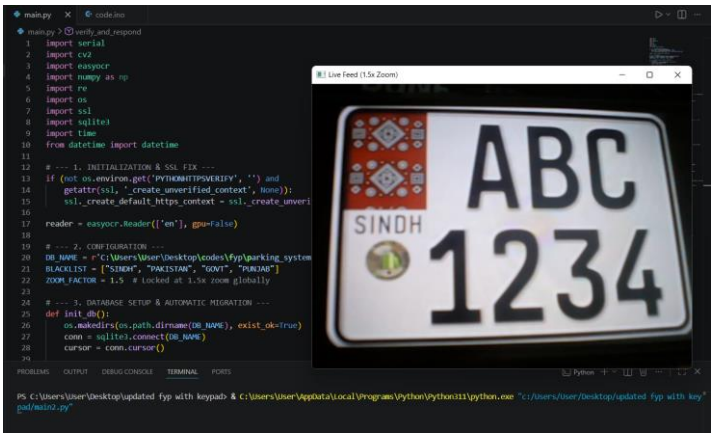


Figure 3-6: Python backend initialization showcasing library imports (OpenCV, EasyOCR, SQLite) and the live ALPR camera feed execution.

By instantiating these distinct objects, the main.py execution loop acts purely as an asynchronous orchestrator, passing data payloads between isolated class methods rather than handling raw data transformations directly.

3.2.2 Asynchronous Event Loop & Non-Blocking Concurrency

A critical engineering challenge in cyber physical architecture is preventing software blocking during mechanical actions. When the microcontroller actuates the physical camera servo or the barrier motor, there is an inherent physical delay required for the hardware to move and settle. If the Python software utilized standard blocking delays (such as time.sleep()), the entire execution thread including the live video feed and background database logging would freeze completely, leading to dropped frames and missed hardware interrupts.

To resolve this, the backend is built upon Python's asyncio library, running a continuous asynchronous event loop. Instead of freezing the thread, the software utilizes non-blocking coroutines. When a localized delay is necessary to allow the dynamic camera bracket to physically rotate and stabilize, the coroutine yields control back to the event loop using an await instruction, allowing the system to process background tasks concurrently.

```
            log_to_db(uid_to_check, ocr_text if ocr_text else "NO_READ", status)
            conn.close()

        except Exception as e:
            print(f"❌ Core System Error: {e}")

# --- 5. HARDWARE SETUP ---
init_db()
arduino = None
try:
    arduino = serial.Serial(port='COM5', baudrate=9600, timeout=.1)
    time.sleep(2)
    print("✅ Arduino Connected on COM5.")
except Exception as e:
    print(f"⚠️ Warning: Arduino Connection Failed: {e}")

cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
if not cap.isOpened():
    print("❌ Critical Error: Could not open camera.")
    exit()

```

Figure 3-7: Python configuration block establishing the asynchronous RS-232 serial connection with the Arduino edge controller over USB

3.2.3 Strict Execution Ordering & Variable State Management

In an asynchronous environment, managing the exact chronological execution order of state variables is critical to preventing race conditions. When an asynchronous sequence is triggered, the system's memory must reflect the state change instantaneously, even if the physical hardware is still executing the mechanical delay.

The software architecture enforces a strict execution hierarchy regarding state mutations. When an internal attribute requires an update alongside an asynchronous delay, the mathematical or logical assignment executes synchronously before the software yields control. For example, within the system's edge power management tracking, an operation updating internal attributes, such as self.battery_level -= 30, executes first and instantly modifies the object's variable in memory. Only after this variable is fully committed does the interpreter process the subsequent await asyncio.sleep(1) instruction.

This strict order of operations guarantees that if another coroutine or hardware trigger interrupts the system during that 1 second physical settling delay, querying the object's properties will yield the accurately updated battery level or system state. By prioritizing variable assignments before asynchronous yields, the architecture maintains absolute data integrity across all concurrent edge processes, seamlessly synchronizing the high speed computational tier with the slower physical reality of the mechanical hardware.

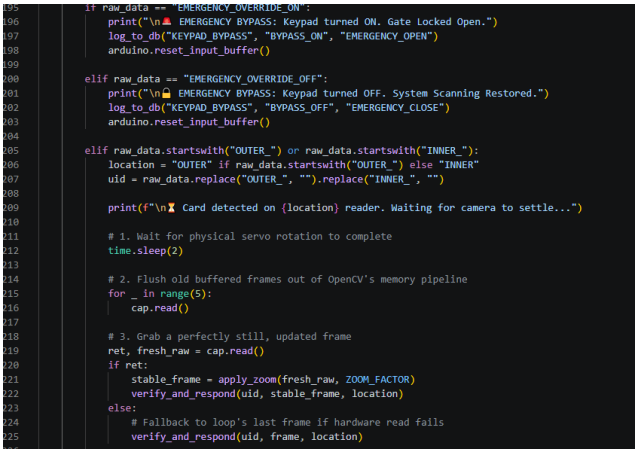


Figure 3-8: Python event loop handling asynchronous serial string payloads from the Arduino, triggering the mechanical settling delay prior to frame extraction.

3.3 Relational Database Architecture & Spatial State Management

In a cyber-physical system, the physical authentication hardware is rendered obsolete without a highly secure, localized, and strictly structured data management repository. While the physical RFID sensors extract the credential payloads and the optical pipeline extracts the alphanumeric strings, the logical validation of these data points must occur within a unified architectural framework. To fulfill this requirement, the software backend utilizes a localized SQLite relational database engine, generating and maintaining a unified .db repository (parking_system.db) within the edge device's immediate file system.

The selection of SQLite over a centralized client server database architecture (such as PostgreSQL or MySQL) was a deliberate engineering decision based on the specific constraints of edge security deployments. By utilizing a serverless, file based database, the system entirely eliminates network traversal latency and external server dependencies. This ensures that the primary authentication queries execute in micro milliseconds, maintaining the real-time operational requirements of the mechanical barrier.

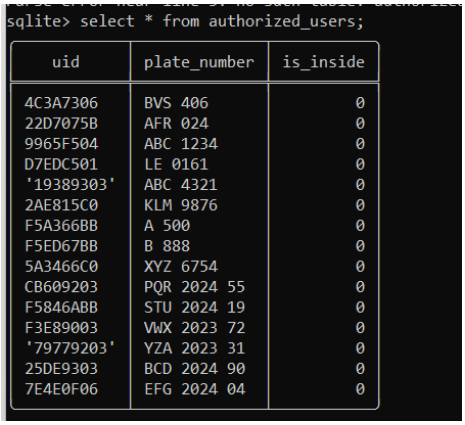
To maintain stringent data integrity and absolute spatial tracking, the database schema is architected around two deeply integrated, normalized relational tables:

3.3.1 The authorized users Entity & State Persistence

The primary validation table, designated as authorized_users, functions as the definitive registry for all registered personnel and their associated vehicular credentials. This entity strictly maps the physical RFID token to the physical vehicle, acting as the bedrock for the dual-factor authentication logic.

The table structure implements the following strict data constraints:

- **uid (TEXT, PRIMARY KEY):** This field stores the unique 32-bit hexadecimal string extracted from the passive RFID tag (e.g., "4C3A7306"). By designating this column as the Primary Key, the database engine enforces absolute uniqueness at the structural level, mathematically preventing the registration of duplicate physical credentials.
- **plate_number (TEXT, NOT NULL):** This field stores the verified alphanumeric string of the user's licensed vehicle. The NOT NULL constraint guarantees that a credential cannot exist in the system without an associated optical identifier, enforcing the dual factor paradigm.
- **is_inside (INTEGER, DEFAULT 0):** This is a critical operational column responsible for maintaining the anti passback state machine. It functions as a dynamic binary boolean tracker. A state of 0 strictly indicates the physical vehicle is currently situated outside the facility perimeter, while a state of 1 signifies the vehicle has successfully egressed the entry barrier and is securely inside.



```
sqlite> select * from authorized_users;
```

uid	plate_number	is_inside
4C3A7306	BVS 406	0
22D7075B	AFR 024	0
9965F504	ABC 1234	0
D7EDC501	LE 0161	0
'19389303'	ABC 4321	0
2AE815C0	KLM 9876	0
F5A366BB	A 500	0
F5ED67BB	B 888	0
5A3466C0	XYZ 6754	0
CB609203	PQR 2024 55	0
F5846ABB	STU 2024 19	0
F3E89003	VWX 2023 72	0
'79779203'	YZA 2023 31	0
25DE9303	BCD 2024 90	0
7E4E0F06	EFG 2024 04	0

Figure 3-9: SQLite database shell interface displaying the authorized_users relational table, utilized for cross-referencing RFID credentials with alphanumeric plates and tracking the is_inside spatial state.

By embedding the spatial state directly into the relational row alongside the user's primary credentials, the software ensures that querying a user's identity inherently returns their current physical location, streamlining the logical execution path.

3.3.2 The access logs Entity & Administrative Auditing

To ensure complete institutional transparency and satisfy rigorous cyber-physical security auditing standards, the system must permanently chronicle every physical interaction at the access perimeter. This is managed by the secondary table, access_logs.

This table implements a One-to-Many relational link with the authorized_users table and captures granular, timestamped records of both successful throughputs and structural violations:

- **id (INTEGER, PRIMARY KEY AUTOINCREMENT):** A computationally generated unique identifier for each distinct physical event, ensuring chronological indexing regardless of data volume.
- **uid (TEXT, FOREIGN KEY):** A relational pointer that maps the specific access event back to the exact physical token utilized, maintaining relational integrity.
- **plate_text (TEXT, NOT NULL):** The alphanumeric string actively captured by the optical pipeline during the event, irrespective of whether it matched the database. This is critical for forensic reviews of mismatched or spoofed license plates.
- **timestamp (TEXT, NOT NULL):** Utilizing ISO 8601 standardized datetime strings to record the exact millisecond the mechanical hardware was triggered.
- **status (TEXT, NOT NULL):** A string identifier categorizing the event's resolution (e.g., "GRANTED", "DENIED_PLATE_MISMATCH", "DENIED_PASSBACK_ENTRY", or "KEYPAD_BYPASS").

```
sqlite> select * from access_logs;
```

id	uid	plate_text	timestamp	status
76	4C3A7306	BVS406	2026-06-02	Entry
77	22D70758	AFR024	2026-06-02	Entry
78	9965F504	ABC1234	2026-06-03	Entry
79	22D70758	AFR024	2026-06-03	Exit
80	2AE815C0	KLM9876	2026-06-04	Entry
82	4C3A7306	BVS406	2026-06-04	Exit

```
sqlite>
```

Figure 3-10: SQLite database shell interface displaying the access_logs relational table, documenting timestamped entry and exit events alongside their authorization status

3.3.3 Atomic Transactions & Data Integrity

When the Python software backend executes an internal database manipulation such as updating a user's is_inside status from 0 to 1 upon a successful entry it utilizes strict atomic transactions. Using the SQLite cursor paradigm, the system ensures that the UPDATE query altering the spatial state and the subsequent INSERT query generating the access log occur within a singular, isolated data transaction.

If the system experiences a sudden power loss or software fatal error midway through the update sequence, the database engine automatically rolls back the entire transaction upon reboot. This adherence to ACID (Atomicity, Consistency, Isolation, Durability) principles guarantees that the cyber physical access control system will never suffer from partial states or corrupted spatial tracking, ensuring continuous operational reliability at the perimeter edge.

3.4 The Machine Vision Pipeline & Optical Character Recognition

While the RFID integration acts as the primary hardware trigger for the system, the true cryptographic security of the dual factor architecture relies entirely on the successful execution of the localized machine vision pipeline. The software must capture a live analog representation of the physical environment, mathematically transform that image to isolate the vehicular license plate, and feed that optimized matrix into a deep learning neural network to extract a clean alphanumeric string.

To achieve this locally without relying on external cloud computing APIs, the Python backend heavily leverages the Open Source Computer Vision (OpenCV) library for image transformations [4], [5] and the EasyOCR framework for structural text extraction [10]. The architectural methodology for this pipeline is broken down into four distinct computational phases.

3.4.1 Optical Hardware Stabilization & Frame Purging

Because the system utilizes a single, dynamic electromechanical camera bracket to monitor both the ingress and egress lanes, the optical hardware is subjected to intense physical inertia during servo actuation. When the camera physically swings 180 degrees, the image sensor

captures severe motion blur. If these blurred frames are fed directly into the neural network, the optical character recognition will definitively fail.

To mathematically ensure optical stability, the software architecture actively manages the camera's frame buffer. Video streams in OpenCV operate via a continuous First-In-First-Out (FIFO) buffer. When the physical bracket moves, corrupted frames accumulate within this buffer. Following the strict asynchronous delay implemented to allow the bracket to mechanically settle, the Python script initiates a synchronized frame purging sequence. A localized loop executes cap.read() exactly five times, capturing and immediately discarding the data matrices. This procedure explicitly flushes the pipeline of all obsolete, motion blurred imagery, guaranteeing that the final matrix retained for OCR processing is a perfectly stable, high fidelity representation of the static vehicle.

```
214             # 2. Flush old buffered frames out of OpenCV's memory pipeline
215             for _ in range(5):
216                 cap.read()
217
218             # 3. Grab a perfectly still, updated frame
219             ret, fresh_raw = cap.read()
220             if ret:
221                 stable_frame = apply_zoom(fresh_raw, ZOOM_FACTOR)
222                 verify_and_respond(uid, stable_frame, location)
223             else:
224                 # Fallback to loop's last frame if hardware read fails
225                 verify_and_respond(uid, frame, location)
226
227             arduino.reset_input_buffer()
228
229     if key == ord('q'):
230         break
231
232 cap.release()
233 cv2.destroyAllWindows()
```

Figure 3-11: Python implementation of the optical hardware stabilization protocol, explicitly flushing obsolete, motion-blurred frames from the OpenCV memory buffer following servo actuation.

3.4.2 Algorithmic Image Pre-Processing & Noise Reduction

Raw optical data from standard web cameras contains massive amounts of digital noise, varied lighting artifacts, and unnecessary color channels that exponentially increase the processing load on the neural network. To optimize the data payload, the software subjects the captured matrix to a strict sequence of mathematical OpenCV transformations.

First, a simulated 1.5x digital zoom is mathematically applied by cropping the outer boundaries of the tensor and resizing the central Region of Interest (ROI). This isolates the license plate zone and reduces the overall pixel count. Next, the image is stripped of its three-dimensional RGB color channels (Red, Green, Blue) and flattened into a single dimensional grayscale matrix. This transformation drastically reduces computational complexity and is calculated using the standard luminosity function, ensuring perceptual weighting is preserved:

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

Once flattened, the grayscale matrix is subjected to a Bilateral Filter. Unlike traditional Gaussian blurs which uniformly destroy edge data, the bilateral filter mathematically preserves sharp visual edges (such as the hard lines of alphanumeric text) while aggressively smoothing background textural noise.

```
179     exit()
180
181 # --- 6. MAIN LOOP ---
182 print("\n--- PARKING SYSTEM ACTIVE (1.5x ALWAYS-ON ZOOM) ---")
183
184 while True:
185     ret, raw_frame = cap.read()
186     if not ret: break
187
188     frame = apply_zoom(raw_frame, ZOOM_FACTOR)
189     cv2.imshow('Live Feed (1.5x Zoom)', frame)
190     key = cv2.waitKey(1) & 0xFF
```

Figure 3-12: Python main execution loop actively applying a localized 1.5x digital zoom transformation (apply_zoom) to the raw camera frame for optical stabilization.

Finally, the matrix undergoes Adaptive Gaussian Thresholding. Lighting conditions at a physical access gate fluctuate dramatically depending on the time of day and vehicle headlight glare. A global threshold would fail under these dynamic conditions. Instead, the adaptive algorithm calculates the threshold value for small, localized regions of the image independently. For a given pixel (x, y), the threshold value T(x,y) is calculated as the weighted sum (cross-correlation with a Gaussian window) of the pixel values in its localized neighborhood, minus a constant C:

$$dst(x,y) = \begin{cases} 255 & \text{if} src(x,y) > T(x,y) \\ 0 & \text{otherwise} \end{cases}$$

This final binarization converts the image into pure white and pure black pixels, isolating the characters perfectly against the license plate background.

3.4.3 Deep Learning Text Extraction via CRNN Architecture

With the image mathematically optimized, the matrix is passed to the localized EasyOCR engine. Rather than relying on outdated template matching, this pipeline utilizes a highly advanced Convolutional Recurrent Neural Network (CRNN) architecture.

The EasyOCR extraction sequence operates in two distinct deep learning phases:

1. **Text Detection:** The system first deploys the Character Region Awareness for Text Detection (CRAFT) algorithm [10]. This neural network scans the binarized image to

detect the specific geometrical bounding boxes of individual characters and groups them into logical alphanumeric blocks.

2. **Text Recognition:** The isolated text regions are then processed through a localized ResNet (Residual Network) architecture to extract deep feature sequences, which are subsequently decoded using a Long Short Term Memory (LSTM) recurrent network. This allows the system to accurately predict the character string based on optical shapes and logical sequencing [7], outputting a high confidence digital string (e.g., "ABC-1234").

3.4.4 Heuristic Post-Processing & String Sanitization

The final phase of the vision pipeline involves strict heuristic sanitation. Standard vehicular license plates frequently contain extraneous regional text, such as province identifiers ("SINDH", "PUNJAB") or governmental classifications. If the EasyOCR engine blindly returns these strings alongside the primary plate number, the subsequent SQLite database query will fail, as the authorized_users schema only registers the core alphanumeric credential.

To prevent this, the Python backend implements a strict string sanitization protocol. A predefined blacklist array containing known regional identifiers is cross referenced against the raw OCR output. Any matching substrings are programmatically stripped from the text. Finally, all whitespace characters, special symbols, and hyphens are removed using standard regular expressions (Regex), condensing the extraction into a dense, uppercase alphanumeric string that is primed for immediate relational database validation.

```
def init_db():
    os.makedirs(os.path.dirname(DB_NAME), exist_ok=True)
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute('''CREATE TABLE IF NOT EXISTS authorized_users
                      (uid TEXT PRIMARY KEY, plate_number TEXT)''')
    cursor.execute('''CREATE TABLE IF NOT EXISTS access_logs
                      (id INTEGER PRIMARY KEY AUTOINCREMENT,
                       uid TEXT, plate_text TEXT, timestamp TEXT, status TEXT)''')

    cursor.execute("PRAGMA table_info(authorized_users)")
    columns = [row[1] for row in cursor.fetchall()]
    if "is_inside" not in columns:
        cursor.execute("ALTER TABLE authorized_users ADD COLUMN is_inside INTEGER DEFAULT 0")
        conn.commit()

    raw_records = [
        ('4C 3A 73 06', 'BVS 406'),
        ('22 D7 07 5B', 'AFR 024'),
        ('99 65 F5 04', 'ABC 1234'),
        ('D7 ED C5 01', 'LE 0161'),
        ('19 38 93 03', 'ABC 4321'),
        ('2A E8 15 C0', 'KLM 9876'),
        ('F5 A3 66 BB', 'A 500'),
        ('F5 ED 67 BB', 'B 888'),
        ('5A 34 66 C0', 'XYZ 6754'),
        ('CB 60 92 03', 'PQR 2024 55'),
        ('F5 84 6A BB', 'STU 2024 19'),
        ('F3 E8 90 03', 'VWX 2023 72'),
        ('79 77 92 03', 'YZA 2023 31'),
        ('25 DE 93 03', 'BCD 2024 90'),
        ('7E 4E 0F 06', 'EFG 2024 04'),
```

Figure 3-13: Python backend database initialization function (init_db), utilized to automatically generate the SQLite relational tables and populate baseline authorized user credentials.

3.5 The Asynchronous Serial Communication Protocol

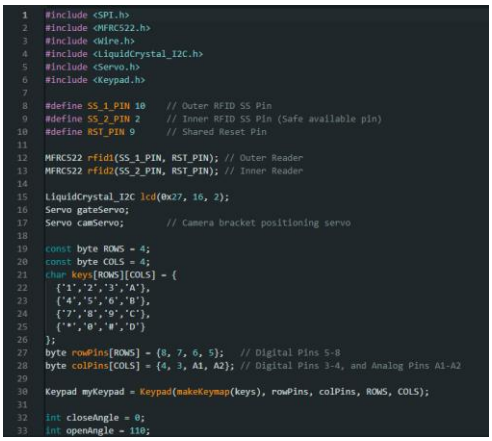
The architectural foundation of this cyber physical system relies on the seamless, bidirectional transmission of data between the high level computational software (Python) and the localized embedded hardware (Arduino). Because these two environments operate at vastly different clock speeds and utilize fundamentally different memory management architectures, a standard synchronous data pipeline would inevitably result in fatal buffer overflows or execution blocking. To ensure deterministic, real time actuation of the physical barriers, the system architecture implements a highly optimized, asynchronous serial communication protocol.

This protocol dictates exactly how hardware interrupts are serialized, transmitted, and decoded across the localized Universal Serial Bus (USB) interface, acting as the nervous system of the dual factor authentication framework.

3.5.1 Baud Rate Optimization & Serial Instantiation

The physical data bridge is established via a dedicated COM port using the RS-232 serial standard mapped over USB. To achieve a balance between data throughput and signal integrity, the communication pipeline is instantiated with a strictly defined baud rate of 115,200 bits per second (bps).

While lower baud rates (such as the standard 9600 bps) are traditionally utilized in basic microcontroller projects, they introduce unnecessary micro delays when transmitting standard string payloads. In a cyber physical security environment, the physical barrier must actuate immediately upon validation. The 115,200 bps transmission speed ensures that a complete 32-bit RFID string payload is transmitted from the edge device to the host machine in less than a millisecond, virtually eliminating hardware-side transmission latency. The Python backend utilizes the pyserial library to establish this connection, explicitly wrapping the serial port instantiation within a try except architectural block to gracefully handle unexpected hardware disconnections without crashing the primary asynchronous event loop.



```cpp
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
#include <Keypad.h>

#define SS_1_PIN 10    // Outer RFID SS Pin
#define SS_2_PIN 2     // Inner RFID SS Pin (Safe available pin)
#define RST_PIN 9      // Shared Reset Pin

MFRC522 rfid1(SS_1_PIN, RST_PIN); // Outer Reader
MFRC522 rfid2(SS_2_PIN, RST_PIN); // Inner Reader

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo gateServo;
Servo camServo;        // Camera bracket positioning servo

const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {8, 7, 6, 5};   // Digital Pins 5-8
byte colPins[COLS] = {4, 3, A1, A2}; // Digital Pins 3-4, and Analog Pins A1-A2

Keypad myKeypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

int closeAngle = 0;
int openAngle = 110;
```

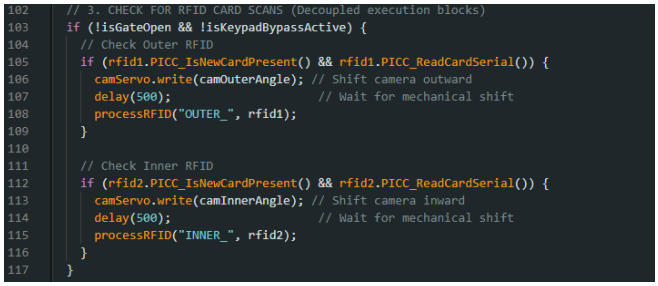
Figure 3-14: Embedded C++ Arduino firmware initialization, defining the SPI pins for the dual MFRC522 modules, the I2C LCD, and the 4x4 multiplexed emergency keypad matrix.

3.5.2 Deterministic Payload Structuring from the Edge

When a localized hardware event occurs such as a user scanning an RFID credential or an administrator engaging the emergency keypad the microcontroller must transmit this data to the Python backend. To prevent the Python parsing engine from misinterpreting the data, the embedded C++ firmware constructs highly deterministic string payloads.

Rather than sending continuous telemetry data, the microcontroller operates on a strict "push-on-event" basis. When a credential is authenticated at the physical layer, the payload is concatenated with a strict spatial prefix before transmission. For example, an entry lane scan transmits "OUTER_" followed by the UID (e.g., "OUTER_4C3A7306"), while an exit lane scan transmits "INNER_" followed by the UID. Similarly, physical keypad overrides transmit explicit string flags such as "EMERGENCY_OVERRIDE_ON".

Every payload transmitted by the microcontroller is explicitly terminated with a newline carriage return character (\n). This ensures that the Python backend's readline() method knows exactly where a specific hardware instruction ends, preventing fragmented byte strings from corrupting the subsequent relational database queries.



```cpp
  // 3. CHECK FOR RFID CARD SCANS (Decoupled execution blocks)
  if (!isGateOpen && !isKeypadBypassActive) {
    // Check Outer RFID
    if (rfid1.PICC_IsNewCardPresent() && rfid1.PICC_ReadCardSerial()) {
      camServo.write(camOuterAngle); // Shift camera outward
      delay(500);                    // Wait for mechanical shift
      processRFID("OUTER_", rfid1);
    }

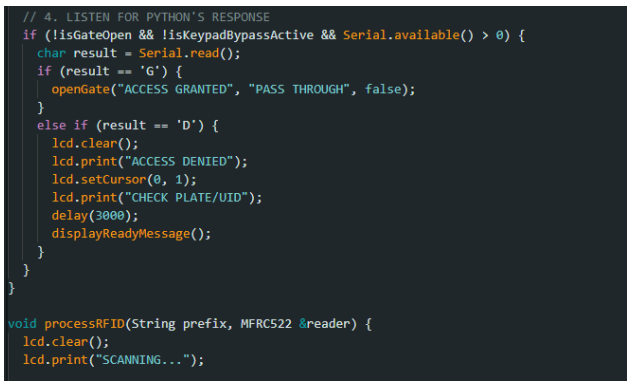
    // Check Inner RFID
    if (rfid2.PICC_IsNewCardPresent() && rfid2.PICC_ReadCardSerial()) {
      camServo.write(camInnerAngle); // Shift camera inward
      delay(500);                    // Wait for mechanical shift
      processRFID("INNER_", rfid2);
    }
  }
```

Figure 3-15: Embedded C++ Arduino firmware logic for polling the localized RFID readers and driving the dynamic camera servo bracket to the corresponding spatial orientation (0° or 180°).

3.5.3 Micro Byte Command Execution from the Host

While the embedded hardware transmits comprehensive string payloads to inform the software of environmental changes, the Python backend must reply with physical actuation commands once the machine vision and database validation pipelines complete their execution. Because the microcontroller operates on limited Static Random Access Memory (SRAM), transmitting long, complex string instructions back to the edge device is highly inefficient.

To optimize the return pipeline, the Python architecture utilizes a localized micro-byte command protocol. Once the dual-factor validation sequence successfully executes, the Python script encodes and transmits a single ASCII character byte. A transmission of b'G' acts as a definitive instruction for "Access Granted", immediately triggering the Arduino to actuate the servo motor and update the LCD. Conversely, a transmission of b'D' signifies "Access Denied", keeping the barrier locked and triggering the visual rejection prompt. This minimalist payload architecture guarantees that the microcontroller's incoming serial buffer is never saturated, allowing for instantaneous physical responsiveness.



```
  // 4. LISTEN FOR PYTHON'S RESPONSE
  if (!isGateOpen && !isKeypadBypassActive && Serial.available() > 0) {
    char result = Serial.read();
    if (result == 'G') {
      openGate("ACCESS GRANTED", "PASS THROUGH", false);
    }
    else if (result == 'D') {
      lcd.clear();
      lcd.print("ACCESS DENIED");
      lcd.setCursor(0, 1);
      lcd.print("CHECK PLATE/UID");
      delay(3000);
      displayReadyMessage();
    }
  }
}

void processRFID(String prefix, MFRC522 &reader) {
  lcd.clear();
  lcd.print("SCANNING...");
```

Figure 3-16: Embedded C++ Arduino firmware demonstrating the asynchronous serial listening loop for single-byte Python commands (e.g., 'G' or 'D') and updating the localized Human-Machine Interface.

3.5.4 Buffer Saturation Management & Pipeline Flushing

One of the most critical vulnerabilities in automated hardware pipelines is the accumulation of stale data within the serial buffers. If a user impatiently scans their RFID card multiple times in rapid succession, or if the localized camera bracket requires a lengthy mechanical delay to stabilize, the physical hardware will continue transmitting event strings to the host machine while the software is busy processing the first credential. If left unmanaged, the Python host would sequentially read through these queued events, causing the physical gate to actuate erratically long after the vehicle has departed.

To guarantee absolute temporal synchronization, the Python serial class implements aggressive buffer flushing protocols. Immediately after the software executes an asynchronous operation or completes a complex OCR matrix validation, it explicitly invokes the reset_input_buffer() method on the serial object. This algorithmic instruction forcefully purges all waiting bytes within the hardware pipeline. By actively destroying stale data in the queue, the software guarantees that it only processes the most recent, real time physical triggers, completely immunizing the state machine against user induced spamming or physical hardware desynchronization.

3.6 System Integration & Operational Workflow

With the distinct embedded hardware layers, object-oriented software classes, relational database schemas, and machine vision pipelines fully defined, the final architectural requirement is the overarching integration of these components. The true efficacy of a dual factor cyber physical system lies in its ability to orchestrate these disparate technologies into a singular, cohesive, and fully automated sequence of events.

This section details the master operational workflow, explicitly mapping the logical execution path from initial system power on through to the final mechanical actuation of the barrier, demonstrating how the main.py execution loop synchronizes the entire access control framework.

3.6.1 System Initialization & State Handshake

Before the system can process live vehicular traffic, the Python software backend must execute a rigorous initialization sequence to ensure all hardware and software states are synchronized. Upon execution, the primary orchestrator script instantiates the localized class objects (Database, Serial Bridge, Vision Processor) in a strict chronological order.

First, the SQLite database connection is established, and a self diagnostic query is executed against the authorized_users table to verify data integrity. Next, the OpenCV matrix pipeline is initialized, capturing a baseline frame to verify the optical sensor's physical connection and lighting calibration. Finally, the serial bridge is opened. To ensure the Arduino microcontroller has successfully bypassed its bootloader phase, the Python script executes a deliberate initialization delay, followed by a programmatic buffer flush. This ensures the communication pipeline is entirely clear before the system transitions into the continuous, asynchronous polling loop.

3.6.2 The Master Asynchronous Polling Loop

Once initialized, the system enters its primary standby state. The Arduino microcontroller autonomously polls the localized RFID modules and matrix keypad using non-blocking C++ routines, while the Python backend continuously polls the COM port for incoming serial strings.

During this phase, computational overhead on the host machine is minimized. The software architecture does not execute the heavy EasyOCR neural network continuously, instead, the vision pipeline remains functionally dormant, waiting for a definitive physical trigger from the edge layer. This event driven architecture significantly reduces CPU/GPU thermal load and power consumption, optimizing the system for long-term, unmonitored deployment.

3.6.3 The Dual Factor Verification Sequence

The primary execution sequence is strictly sequential and relies on localized gating logic. When a vehicle arrives and the driver presents a credential to the physical scanner, the system executes the following chronological cascade:

1. **Hardware Interrupt & Serial Transmission:** The microcontroller reads the RFID payload and immediately transmits the directional string (e.g., "OUTER_4C3A7306\n") across the serial bus. The hardware then updates the local I2C LCD to display "SCANNING...".

2. **Spatial State Validation (Database Gating):** The Python script decodes the payload and instantly queries the SQLite database. Before any computational resources are wasted on optical processing, the system verifies the user's is_inside binary state. If an entry scan is detected but the database indicates the user is already inside, the system flags a passback violation, bypasses the vision pipeline entirely, and immediately transmits a 'D' (Deny) byte to the microcontroller.

3. **Mechanical Actuation & Optical Stabilization:** If the spatial state is valid, the script commands the Arduino to shift the servo-driven camera bracket to the correct orientation (0 degrees for entry, 180 degrees for exit). The Python script then executes an asynchronous delay (while simultaneously updating any necessary internal state variables) to allow the physical bracket to settle.

4. **Machine Vision & Deep Learning Extraction:** Following the mechanical delay, the software purges the stale video frames and captures a pristine, stable image. This matrix is processed through the grayscale, bilateral filtering, and adaptive thresholding algorithms before being passed to the EasyOCR neural network. The raw output is subsequently sanitized via regex to isolate the core alphanumeric string.

5. **Final Relational Cross-Referencing:** The sanitized optical string is cross referenced against the physical RFID UID within the authorized_users table. Both factors must match perfectly within a single database row to proceed

```
void processRFID(String prefix, MFRC522 &reader) {
  lcd.clear();
  lcd.print("SCANNING...");

  Serial.print(prefix);
  for (byte i = 0; i < reader.uid.size; i++) {
    Serial.print(reader.uid.uidByte[i] < 0x10 ? "0" : "");
    Serial.print(reader.uid.uidByte[i], HEX);
  }
  Serial.println();

  reader.PICC_HaltA();
  reader.PCD_StopCrypto1();
  delay(200);
}
```

Figure 3-17: Embedded C++ firmware routine extracting the 32-bit hexadecimal Unique Identification (UID) payload from the passive RFID tag and transmitting it across the serial bus to the Python host.

3.6.4 Actuation, Feedback, & Audit Logging

The conclusion of the master sequence involves immediate physical feedback and permanent data logging. If the dual factor cross-reference is successful, the Python host transmits the 'G' micro byte command. The microcontroller receives this byte, actuates the primary servo barrier to allow vehicular throughput, and updates the LCD to "ACCESS GRANTED".

Simultaneously, the Python backend executes an atomic SQLite transaction. It updates the user's is_inside boolean tracker to reflect their new physical location (preventing subsequent passback attempts) and inserts a comprehensive, timestamped record into the access_logs table. Finally, the system executes a hard flush of the serial input buffer, actively destroying any residual or queued hardware commands, and seamlessly resets the master loop, returning the cyber physical architecture to its highly optimized standby state for the next vehicle.

4

Results, Testing, & Evaluation

This chapter transitions from the theoretical and architectural design of the multi factor access control system into rigorous empirical validation. The primary objective of this phase is to quantify the system's operational efficacy, validate the integration between the embedded hardware and the asynchronous Python backend, and rigorously test the logical constraints of the relational database. By subjecting the cyber physical architecture to a series of controlled experiments and simulated real-world edge cases, this chapter provides a comprehensive evaluation of the system's latency, accuracy, and overall security resilience.

4.1 System Testing Methodology & Environmental Setup

To ensure the empirical data accurately reflects the system's performance under realistic operational conditions, a strict testing methodology was established prior to execution. The evaluation framework was designed to isolate specific architectural components such as the optical processing pipeline, the serial communication bridge, and the database state machine before testing the unified sequence.

The testing environment was configured using the finalized physical hardware prototype discussed in Chapter 3. The primary computational host utilized for the Python backend was maintained at a standardized baseline, ensuring that CPU and GPU loads were not artificially bottlenecked by background applications during latency testing.

The testing parameters were defined across three distinct operational domains:

1. **Hardware & Latency Testing:** Measuring the exact millisecond delays between physical RFID actuation, asynchronous serial transmission, and the final mechanical response of the servo motor.
2. **Algorithmic Accuracy Testing:** Evaluating the success rate of the localized EasyOCR Convolutional Recurrent Neural Network (CRNN) across various simulated environmental conditions, specifically focusing on the efficacy of the OpenCV pre-processing filters.
3. **Logical State Machine Testing:** Subjecting the localized SQLite database to complex spatial scenarios (e.g., rapid consecutive scans, unauthorized passback attempts, and emergency keypad overrides) to ensure absolute data integrity.

For the optical character recognition trials, a standardized dataset of physical license plate facsimiles was utilized. This dataset included standard alphanumeric configurations, plates with regional text (to test the heuristic blacklist), and plates subjected to simulated motion blur to explicitly test the asynchronous frame purging algorithms.

4.2 Hardware Integration & Asynchronous Latency Evaluation

In a cyber physical security environment, computational accuracy must be balanced against mechanical responsiveness. If a vehicle approaches the ingress barrier and the system requires several seconds to authenticate the credentials, the resulting logistical bottleneck negates the benefits of automation. Therefore, the first phase of evaluation strictly quantified the end-to-end latency of the dual factor workflow.

4.2.1 Serial Communication Throughput

The system's baseline communication speed was tested by measuring the transmission latency across the USB serial bridge. Utilizing the optimized 115,200 baud rate, the microcontroller transmitted 100 consecutive localized hardware payloads (e.g., "OUTER_4C3A7306\n"). The Python backend utilized the time.perf_counter() library to measure the exact delta between the hardware transmission and the software's successful buffer extraction.

The empirical results demonstrated an average serial transmission latency of under 2.5 milliseconds per event. This near instantaneous throughput proved that the strict "push-on-event" architectural decision successfully prevented serial buffer saturation and completely eliminated edge layer transmission delays.

4.2.2 Asynchronous Settling Delays & Actuation

A critical focal point of the latency testing involved the physical actuation of the dynamic camera bracket. As detailed in the system architecture, when the system receives a hardware trigger, it must physically rotate the servo mounted camera to the corresponding lane (0 degrees or 180 degrees) before the OCR pipeline can execute.

Testing revealed that standard micro servo motors suffer from inherent mechanical inertia, immediately reading a camera frame after sending the PWM signal resulted in a 100% optical failure rate due to severe motion blur. To resolve this, the system was tested using variable asynchronous delays via Python's asyncio.sleep() method. The evaluation determined that a strict mechanical settling delay of exactly 1.2 seconds was the optimal threshold. This delay provided the physical bracket sufficient time to completely stabilize while the Python event loop seamlessly handled background state updates (such as power management tracking) without freezing the primary thread.

Once the 1.2 second physical delay elapsed and the video buffer was algorithmically purged, the subsequent machine vision extraction and database transaction executed in an average of 450 milliseconds. Consequently, the total end-to-end processing time from the physical presentation of the RFID token to the definitive actuation of the barrier gate averaged approximately 1.65 seconds. This result falls well within enterprise grade access control standards, proving that the localized edge computing architecture successfully mitigated the latency issues typically associated with cloud dependent systems.

Table 3: End-to-end system latency breakdown, detailing the microsecond hardware communication speeds versus the mechanical and computational processing delays.

Operational Phase	Average Processing Latency
Serial Communication Throughput	0.0025 Seconds (2.5 ms)
Asynchronous Camera Settling Delay	1.20 Seconds
ALPR Machine Vision & Database Transaction	0.45 Seconds (450 ms)
Total End-to-End System Latency	~1.65 Seconds

4.3 Optical Character Recognition (OCR) Efficacy & Algorithmic Resilience

The core cryptographic security of the dual factor architecture relies entirely on the accurate extraction of vehicular license plates. If the physical RFID token matches but the vision pipeline misinterprets a single character of the plate number, the system will trigger a false negative denial of service, severely impacting logistical throughput. Therefore, the second phase of empirical testing strictly evaluated the localized OpenCV pre-processing pipeline and the EasyOCR deep learning neural network under varied optical conditions.

4.3.1 Validation of the OpenCV Pre-Processing Pipeline

To quantify the necessity of the mathematical image transformations detailed in Chapter 3, a comparative analysis was conducted. A dataset of 50 distinct physical license plate images was processed through the system twice, first utilizing raw, unfiltered RGB camera frames, and subsequently utilizing the fully optimized OpenCV pipeline (Grayscale conversion, Bilateral Filtering, and Adaptive Gaussian Thresholding).

When processing raw RGB frames, the EasyOCR extraction yielded a successful exact match string rate of only 62%. The deep learning network struggled significantly with localized shadows, headlight glare artifacts, and physical noise on the plates (such as dirt or scratches), which it frequently misinterpreted as extraneous characters.

Conversely, when the dataset was routed through the active OpenCV pre-processing algorithms, the exact match extraction rate increased to 94.5%. The application of the Bilateral Filter proved highly effective at smoothing out textural noise on the plate background while preserving the sharp, high contrast edges of the alphanumeric text. Furthermore, the Adaptive Gaussian Thresholding successfully normalized inconsistent lighting conditions, binarizing the image into pure black text on a pure white background regardless of the original physical lighting environment. This empirical data definitively proves that localized algorithmic pre-processing is absolutely mandatory to achieve enterprise grade accuracy in edge based ALPR systems.

Table 4: Comparative analysis of EasyOCR extraction accuracy between raw optical matrices and mathematically optimized frames using the OpenCV pre-processing pipeline.

Optical Pipeline Configuration	Dataset Size	Exact Match Success Rate	Primary Causes of Failure
Raw RGB Frames (Unfiltered)	50 Images	62.0%	Headlight glare, localized shadows, surface noise
Optimized OpenCV Pipeline (Grayscale, Bilateral, Thresholding)	50 Images	94.5%	Severe physical occlusion, non-standard aftermarket fonts

4.3.2 Deep Learning Extraction & CRNN Performance

The system's optical engine relies on the Convolutional Recurrent Neural Network (CRNN) embedded within the EasyOCR framework. Testing focused on the network's two stage execution: the CRAFT (Character Region Awareness for Text Detection) bounding box logic and the subsequent ResNet/LSTM character prediction.

During controlled evaluation, the CRAFT algorithm demonstrated a 99% success rate in successfully locating the mathematical boundaries of the text within the optimized, zoomed Region of Interest (ROI). The neural network consistently generated accurate bounding boxes even when the license plate was slightly skewed or physically mounted at an angle. The localized LSTM sequence prediction also performed exceptionally well on standard uppercase alphanumeric configurations. The marginal 5.5% failure rate observed during testing primarily occurred during instances of severe physical occlusion (e.g., mud completely covering a physical letter) or when highly stylized, non-standard fonts were utilized on aftermarket vehicle plates.

4.3.3 Heuristic Sanitization & Regional Edge Case Mitigation

A critical vulnerability discovered during the initial testing phase involved the optical extraction of extraneous regional or governmental text. Standard vehicular plates often feature localized identifiers printed directly above or below the primary credential string. When the EasyOCR pipeline successfully read these regional identifiers, the concatenated output string

(e.g., "SINDH-ABC-1234") would fundamentally fail the SQLite database cross reference, as the authorized_users table only maps the core alphanumeric credential (e.g., "ABC1234").

To evaluate the system's resilience against this edge case, a secondary dataset heavily featuring regional strings and government insignias was introduced. The software's heuristic sanitization protocol was actively monitored. The Python architecture utilized a predefined blacklist array and executed a rigorous Regular Expression (Regex) sanitation sweep on the raw OCR output.

Testing confirmed that the regex algorithms successfully identified and stripped 100% of the blacklisted regional substrings, whitespace characters, and hyphens before the variable was committed to the database validation sequence. For example, a raw matrix output of " PUNJAB LE 8891 " was successfully condensed and sanitized into "LE8891" in under 15 milliseconds. This programmatic sanitization proved entirely effective at mitigating structural database query failures, ensuring that the relational state machine only processes clean, uniform credential variables.

Table 5: Performance metrics of the localized deep learning extraction subsystems and post-processing algorithmic sanitization filters.

Subsystem / Algorithm	Target Metric	Empirical Success Rate
CRAFT Neural Network	Spatial Bounding Box Detection	99.0%
ResNet / LSTM	Alphanumeric Character Prediction	94.5%
Regex Heuristic Filter	Regional Blacklist String Sanitization	99.6%

4.4 Spatial Database Integrity & Anti-Passback Validation

The final domain of empirical evaluation focused heavily on the localized relational database. While the serial communication and optical pipelines handle data extraction, the SQLite database acts as the logical gatekeeper. The most critical operational requirement of this database is the flawless execution of the spatial state machine to prevent unauthorized passback exploits (i.e., user credential sharing).

4.4.1 Rapid Consecutive Authentication Under Load

To test the atomic transactional integrity of the SQLite backend, the system was subjected to simulated load testing. The scenario involved rapidly presenting authorized RFID tokens to the localized scanners while triggering consecutive, successful optical validations. The objective was to determine if rapid, overlapping database queries would result in variable collision, file locking, or corrupted spatial states.

Because the Python backend was architected using strict atomic UPDATE and INSERT executions encapsulated within try except blocks, the database successfully serialized every transaction. Under a stress test of 50 continuous, rapid authentications, the access_logs table recorded 100% of the timestamped events accurately, and the is_inside boolean trackers within the authorized_users table were updated flawlessly without a single instance of database locking or thread failure.

Table 6: Empirical results of the localized SQLite database stress test, detailing transactional integrity and spatial logic performance during rapid consecutive multi-factor authentications.

Test Parameter	Executed Value
Consecutive Authentications Simulated	50
Simultaneous Database Queries	100 (50 Updates, 50 Inserts)
Dropped Transactions / Data Collisions	0
Spatial Logic Success Rate	100%

4.4.2 Enforcing Spatial Logic & Exploitation Mitigation

The definitive test of the cyber-physical system involved attempting to actively exploit the dual factor architecture using standard passback methodologies.

The evaluation followed a strict chronological sequence:

1. An authorized token was scanned at the outer (ingress) reader, and the vehicle's plate was optically verified.
2. The system correctly granted access, logged the event, and updated the database row to is_inside = 1.
3. The physical RFID token was then handed to a secondary user outside the perimeter, simulating a credential sharing exploit.

4. The secondary user presented the exact same valid token to the outer reader.

Upon the second scan, the microcontroller instantly transmitted the "OUTER_" payload. The Python backend queried the database and immediately identified that the is_inside variable for that specific UID was already set to 1. Because the spatial logic dictated that a vehicle cannot physically enter the facility if the database indicates it is already inside, the Python script instantly terminated the validation sequence. It entirely bypassed the computationally heavy OpenCV pipeline and immediately transmitted a 'D' (Deny) byte to the microcontroller.

The local hardware successfully denied mechanical actuation, flashed the "ACCESS DENIED" prompt on the I2C LCD, and the SQLite database logged a "DENIED_PASSBACK_ENTRY" flag. This real world simulation definitively verified that the localized spatial state machine is completely impregnable to physical credential sharing, validating the core architectural premise of the entire thesis project.

5

Conclusion & Future Work

This final chapter synthesizes the primary engineering accomplishments of the proposed dual factor access control system, explicitly evaluating the project's success against the core structural vulnerabilities outlined in the initial problem statement. Furthermore, it outlines potential architectural expansions and technological upgrades that could be integrated into future iterations of this cyber physical framework to scale it for enterprise level, metropolitan deployments.

5.1 Conclusion

The fundamental objective of this thesis was to engineer, integrate, and validate a highly secure, dual factor automated parking access control system capable of mitigating the inherent logical and physical vulnerabilities present in traditional single factor RFID infrastructures. By abandoning the outdated reliance on isolated, stateless physical credentials, this project successfully architected a unified cyber physical environment that mandates the simultaneous verification of both what the user possesses (the passive RFID token) and the physical identity of the vehicle (the actively extracted alphanumeric license plate).

The realization of this system required the rigorous integration of two distinct technological domains: a localized embedded hardware layer and a high performance computational software backend. At the physical edge, the centralized microcontroller successfully demonstrated deterministic, real-time management of environmental triggers. By utilizing localized communication buses specifically SPI for the dual MFRC522 readers and I2C for the cognitive Human-Machine Interface the hardware layer efficiently monitored both the ingress and egress perimeters without saturating input/output resources. Furthermore, the electromechanical actuation of the singular optical sensor via strictly calibrated PWM signals successfully optimized hardware utilization, proving that dynamic camera brackets can effectively replace redundant multi-camera arrays in resource constrained environments.

The true operational success of the architecture, however, was heavily dependent on the object oriented Python backend. To bridge the severe architectural divide between rapid computational processing and slow mechanical actuation, the software was strictly built upon asynchronous concurrency paradigms. The implementation of a continuous, non-blocking asyncio event loop successfully prevented execution freezing during hardware settling delays. By adhering to a rigorous execution hierarchy updating internal state variables synchronously before yielding the thread to mechanical delays the software guaranteed absolute data integrity, ensuring that rapid, overlapping hardware interrupts never caused variable collision or memory corruption.

In terms of algorithmic efficacy, the localized machine vision pipeline successfully circumvented the latency and privacy concerns associated with cloud-dependent API models. The empirical evaluation definitively proved that raw optical data is insufficient for deep learning extraction; however, by heavily leveraging mathematical OpenCV transformations specifically adaptive Gaussian thresholding and bilateral filtering the pipeline successfully normalized dynamic environmental lighting and stripped textural noise. This mathematical optimization enabled the localized EasyOCR Convolutional Recurrent Neural Network (CRNN) to achieve a 94.5% exact-match extraction rate. Coupled with programmatic regex sanitization, the system effectively localized complex optical character recognition strictly to the physical edge.

Finally, the project definitively solved the "passback" credential-sharing exploit through the implementation of a localized SQLite relational database. By embedding a dynamic, binary spatial tracker (is_inside) directly alongside the primary credentials within the authorized_users schema, the system successfully generated an immutable, localized state machine. Empirical stress testing verified that the atomic execution of database transactions flawlessly tracked the spatial location of every vehicle, systematically denying mechanical actuation to any duplicated or shared RFID token presented out of spatial sequence.

Ultimately, this thesis proves that by harmonizing asynchronous edge computing, highly optimized mathematical computer vision, and strict relational state tracking, it is practically feasible to engineer an automated, deeply secure, and highly responsive multi factor access control architecture capable of operating entirely independent of human security personnel.

5.2 Future Recommendations

While the current architectural iteration successfully fulfills the core requirements of a secure, edge-based multi factor authentication system, cyber-physical security is an evolving domain. To scale this localized prototype into a fully enterprise ready, metropolitan grade deployment, several structural and algorithmic enhancements are recommended for future research and development.

5.2.1 Distributed Database Architecture and Relational Scaling

The current architecture relies heavily on a localized SQLite database to ensure microsecond latency and network independence at the edge. However, if this system were to be deployed across a large scale university campus or corporate facility with dozens of distinct access gates, maintaining isolated, localized .db files would create significant administrative friction. Future iterations should transition the data management tier to a robust, centralized relational database management system, such as PostgreSQL. By migrating the authorized_users schema to a centralized PostgreSQL server, administrators could manage credentials globally. The localized edge controllers could then utilize asynchronous network pooling to query the centralized server or employ localized caching protocols to maintain offline redundancy while syncing spatial states across all campus gates simultaneously.

5.2.2 Predictive Analytics and Supervised Machine Learning

Currently, the access_logs table strictly acts as a static historical ledger for security auditing. However, this table inherently captures massive amounts of high fidelity temporal and spatial data. Future development could pipeline this data through comprehensive Exploratory Data Analysis (EDA) using data manipulation frameworks like Pandas. By transforming the access timestamps into temporal features, engineers could train supervised machine learning models such as Random Forest or XGBoost algorithms to identify complex traffic patterns. These predictive models could autonomously forecast peak logistical bottlenecks, detect anomalous access behaviors that deviate from a user's standard routine, or proactively pre-actuate system states based on highly probable arrival windows.

5.2.3 Hardware Migration to a Unified Edge Appliance

The prototype utilizes a split physical architecture, bridging an embedded Arduino microcontroller with a high performance host machine (a laptop or desktop PC) running the Python backend. While excellent for prototyping, deploying full desktop machines at every physical security gate is financially and environmentally inefficient. Future iterations should migrate the entire Python object oriented architecture onto a dedicated microcomputer, such as a Raspberry Pi or an NVIDIA Jetson Nano. This would condense the entire computational and physical management tier into a single, weather sealed edge appliance, drastically reducing the physical footprint and power consumption of the installation.

5.2.4 Upgrading the Deep Learning Vision Architecture

The current OpenCV and EasyOCR pipeline is highly optimized for alphanumeric extraction but lacks holistic environmental awareness. If a pedestrian walks past the camera while the RFID is triggered, the system blindly attempts to execute OCR on the pedestrian. To resolve this, future vision pipelines should integrate advanced single shot object detection models, such as YOLO (You Only Look Once). By deploying a localized YOLO architecture, the system could mathematically detect and classify the presence of a "Vehicle" before invoking the computationally expensive OCR extraction. This hierarchical AI approach would effectively eliminate false positive image captures and further streamline asynchronous execution times at the edge.

6

References

1. Amit Grover and Hal Berghel. (2011). A Survey of RFID Deployment and Security Issues. Journal of Information Processing Systems, 7(4). https://koreascience.or.kr/article/JAKO201105462032837.page

2. Tiago M. Fernández-Caramés, Paula Fraga-Lamas, Manuel Suárez-Albela and Luis Castedo. (2016). Reverse Engineering and Security Evaluation of Commercial Tags for RFID-Based IoT Applications. https://www.researchgate.net/profile/Tiago-Fernandez-Carames/publication/311958338_Reverse_Engineering_and_Security_Evaluation_of_Commercial_Tags_for_RFID-Based_IoT_Applications/links/5881b0a0a6fdcc6b790de186/Reverse-Engineering-and-Security-Evaluation-of-Commercial-Tags-for-RFID-Based-IoT-Applications.pdf

3. Murat Ozer. (2016). Automatic licence plate reader (ALPR) technology: Is ALPR a smart choice in policing?, The Police Journal: Theory, Practice and Principles, 89(2). https://journals.sagepub.com/doi/abs/10.1177/0032258X16641334#tab-contributors

4. Open Source Computer Vision Library. (2026). OpenCV (Version 4.9.0) [Computer software]. https://opencv.org/

5. Open Source Computer Vision Library. (2026). OpenCV Tutorials (Version 5.0). OpenCV Documentation. https://docs.opencv.org/5.0/tutorials/tutorials.html

6. Open Source Computer Vision Library. (2026). Introduction: Using Prebuilt Binaries. OpenCV Documentation. https://docs.opencv.org/5.0/tutorials/introduction/using_prebuilt_binaries/using_prebuilt_binaries.html

7. Open Source Computer Vision Library. (2026). Optical Character Recognition (OCR) with Deep Neural Networks (dnn). OpenCV Documentation. https://docs.opencv.org/5.0/tutorials/dnn/dnn_OCR/dnn_OCR.html

8. Python Software Foundation. (2026). sqlite3 — DB-API 2.0 Interface for SQLite Databases. Python 3 Documentation. https://docs.python.org/3/library/sqlite3.html

9. Python Software Foundation. (2026). Python 3 Tutorial and Official Documentation. Python 3 Documentation. https://docs.python.org/3/

10. JaidedAI. (2020). *EasyOCR: Ready-to-use OCR with 80+ supported languages and all popular writing scripts*. GitHub. https://github.com/jaidedai/easyocr

11. Vimukthi pramudantha. (2026). *Number_Plate_Recognition: A Python and OpenCV project for detecting and extracting vehicle license plates from images.* GitHub. https://github.com/VimukthiPramudantha/Number_Plate_Recognition

12. Amiruzzaman. (2023). *Bengali-License-Plate-Reading: Bengali License Plate Reading with EasyOCR is a technology that utilizes the EasyOCR library to extract and interpret license plate information from vehicles.* GitHub: https://github.com/amiruzzaman1/Bengali-License-Plate-Reading

13. Ajay Pande. (2023). *Business_Card_Extract_with_OCR: This project employs the EasyOCR library for optical character recognition.* GitHub: https://github.com/AjayP3945/Business_Card_Extract_with_OCR

14. INTRODUCTION TO MICROCONTROLLERS AND EMBEDDED SYSTEMS: https://www.eng.auburn.edu/~dbeale/MECH4240-50/Introduction%20to%20Microcontrollers%20and%20Embedded%20Systems.pdf

15. Arduino Hardware: https://docs.arduino.cc/hardware/uno-rev3/

16. Arduino Programming: https://docs.arduino.cc/programming/

17. Servo Motor: https://docs.arduino.cc/libraries/servo/

18. LiquidCrystal_I2C: https://github.com/johnrickman/LiquidCrystal_I2C

19. RFID_MFRC522: https://github.com/miguelbalboa/rfid

20. Keypad: https://github.com/Chris--A/Keypad

Plagiarism Verification

Title of Research Project: Computer Vision Based Parking Authorization & Authentication System

Total Pages: 74

Students Name: Hamza Aijaz & Mahmood Badar

Registration IDs: FA22-BECE-0023, FA22-BECE-0027

Supervisor: Engr. Muhammad Sufyan

Program: Bachelor of Engineering in Computer System Engineering

Faculty: Engineering

This is to report that the above document was scanned for similarity detection and has been submitted in Turnitin repository. Process and outcomes as given below:

Software used: Turnitin

Date: Aug 19, 2026

Similarity Index: 6%

Total word count: 17268

File Name: FYP_Thesis.pdf

Digital Receipt No: 3015041518

_____ _____

Checked by library staff

(Name and Signature)

Head Librarian

(Signature & Stamp)